Of rotation in the Geneva Code

About the old way of doing, and a possible new one

Cyril Georgy & Sylvia Ekström

April 14, 2016

1 Introduction

This document aims at giving a clear view about how rotation is treated in the current version of the Geneva code (2013), and a proposition on a possible new way of dealing with it, with the hope of an improvement. We will only recall the minimal theoretical background; the interested reader can find all the necessary informations in Maeder (2009). On the other hand, we aim to give as much technical details as needed in order to 1) make clear to the reader what is done, where in the code, and (possibly) why, 2) indicate the approximations that are made at various places, and the possible resulting shortcomings, and finally 3) provide a reference document that will be useful to other people who want to modify/improve the treatment of rotation.

The first part of this document describes the state of the code in its 2013 version, *i.e.* including the modifications for the conservation of angular momentum, mechanical winds when at critical velocity, and anisotropic mass loss by Cyril Georgy (Georgy 2010). This version corrected some "historical" bugs and improved the way angular momentum is conserved during a computation. The price to pay was the introduction of ad-hoc corrections of the angular velocity near the surface, that was not completely satisfactory.

The second part describes a possible new way of dealing with rotation, by: 1) changing the boundary conditions of the differential equations in agreement with other assumptions made inside the code, 2) adding a new method to correct the final angular momentum content of the model, based on a more physical argument, 3) changing the way external fluxes of angular momentum (in case of anisotropic mass loss, mechanical mass loss, magnetic breaking) can be accounted for consistently in the boundary conditions of the advection and diffusion equations. The accuracy of the methods are also discussed, and some test-case are shown.

2 Treatment of rotation: old method

2.1 General philosophy

The problem of the transport of angular momentum in stars basically consist in solving the following equation (*e.g.* Chaboyer & Zahn 1992; Zahn 1992; Maeder & Zahn 1998):

$$\rho\partial_t \left(r^2\Omega\right)\big|_{M_r} = \frac{1}{5r^2}\partial_r\left(\rho r^4\Omega U_2\right) + \frac{1}{r^2}\partial_r\left(\rho D_{\rm v} r^4\partial_r\Omega\right), \tag{1}$$

where ρ is the density, r the radius, Ω the angular velocity, U_2 the vertical component of the meridional circulation field, and $D_{\rm v}$ the vertical diffusion coefficient (due *e.g.* to shear instability)[1]. The

[1]The quantity in this equation have to be understood as mean ones, averaged over an isobar.

boundary conditions are the following (Talon 1997; Maeder 2002):

$$[\partial_r \Omega]_{\text{centre}} = 0, \tag{2}$$

$$[\partial_r \Omega]_{\text{surface}} = \frac{3}{8\pi} \frac{\dot{\mathcal{L}}_{\text{excess}}}{\rho D_{\text{shear}} r^4}, \tag{3}$$

$$\partial_t \left[\Omega \int_{r_t}^{R} r^4 \rho \mathrm{d}r \right] = -\frac{1}{5} r^4 \rho \Omega U_2, \tag{4}$$

$$\partial_t \left[\Omega \int_{r_{\text{inf, CZ}}}^{r_b} r^4 \rho \mathrm{d}r \right] = -\frac{1}{5} r^4 \rho \Omega U_2. \tag{5}$$

In equation (3) $\dot{\mathcal{L}}_{\text{excess}}$ is the excess of angular momentum flux with respect to the isotropic case (Maeder 2002, see below) and reduced to 0 when anisotropic winds are not accounted for, eq. (4) is taken at the top (R being the stellar radius), and eq. (5) at the bottom ($r_{\text{inf, CZ}}$ being the bottom radius of the convective zone below the zone where diffusion/advection is computed).

Two points are important to be noticed here to understand how this equation is actually solved in the Geneva code:

1. Equation (1) is not solved as a whole, but split in its advective (first part of the right-hand side) and diffusive part (second part), and computed separately one time step each. In order to account for that, the diffusion coefficient are multiplied by 2 in the subroutine `coediff`, and the meridional circulation velocity is multiplied by 2 as well in `coediff` and in `giadv`.

2. The resolution of eq. (1) is not solved at the same time as the usual structure equations, but in an iterative process: the structure is solved first, then the transport equation on top of the background of the structure, then the structure is resolved with the new rotation profile, and so on.

Obviously, both points are responsible for some errors in the results, that are (partially) discussed later on.

Another peculiarity of the Geneva code has important consequences on the treatment of rotation: in its design, the code treats separately the so-called "interior", where the full set of structure equations are solved, and the "envelope", where the energy equation is ignored. The envelope is supposed to be homogeneous in chemical composition, and rotating at a constant angular velocity as a solid body.

Finally, in its current implementation, advection is only computed in the first radiative zone below the envelope. This will not be changed in our proposition of modified implementation (see below). So **be aware that in case a model develops a lot of intermediate convective zone, advection will not be applied in each radiative zone, only in the first one starting from the surface.** This also prevent the computation of advection after the main sequence.

2.1.1 Resolution of the diffusion equation

In this section, we show how the diffusive part of the diffusion equation is solved in the Geneva code. The informations presented here are from Meynet et al. (2004). The diffusion equation is:

$$\rho \partial_t \left(r^2 \Omega \right)\big|_{M_r} = \frac{1}{r^2} \partial_r \left(\rho D_{\text{v}} r^4 \partial_r \Omega \right). \tag{6}$$

According to the above "philosophy", we are working on a structure background that is fixed, so we can consider that the radius is not changing (a lot) during the time step. We have thus:

$$\rho r^2 \partial_t \, \Omega|_{M_r} = \frac{1}{r^2} \partial_r \left(\rho D_{\text{v}} r^4 \partial_r \Omega \right). \tag{7}$$

This equation is solved with an implicit (in time) finite difference scheme. Let us multiply this relation by $4\pi r^2$ and integrate it with respect to the radius over a layer centred at a radius r and with a thickness of Δr:

$$\int_{r-\Delta r/2}^{r+\Delta r/2} 4\pi\rho r^4 \partial_t \Omega \mathrm{d}r = \int_{r-\Delta r/2}^{r+\Delta r/2} 4\pi \partial_r \left[\rho D_\mathrm{v} r^4 \partial_r \Omega\right] \mathrm{d}r. \tag{8}$$

With the momentum of inertia of an empty shell of mass $\mathrm{d}m$ and radius r given by:

$$\mathrm{d}\mathcal{I} = \frac{2}{3} r^2 \mathrm{d}m = \frac{2}{3} r^2 4\pi\rho r^2 \mathrm{d}r = \frac{8\pi}{3}\rho r^4 \mathrm{d}r, \tag{9}$$

we thus obtain the following relation:

$$\frac{3}{2}\mathcal{I}\partial_t\Omega = \left[4\pi\rho D_\mathrm{v} r^4 \partial_r \Omega\right]_{r-\Delta r/2}^{r+\Delta r/2}. \tag{10}$$

We following the notations as in Meynet et al. (2004), where the centre of the shells are located at mesh points k and interfaces at k $\pm \frac{1}{2}$, with $\sigma_{\mathrm{k}+\frac{1}{2}} = 4\pi r^4_{\mathrm{k}+\frac{1}{2}} \rho_{\mathrm{k}+\frac{1}{2}}$, and compute the time derivatives between time steps n and n+1:

$$\frac{\Omega_\mathrm{k}^{\mathrm{n}+1} - \Omega_\mathrm{k}^{\mathrm{n}}}{\Delta t} = -\frac{\sigma_{\mathrm{k}+\frac{1}{2}}}{\frac{3}{2}\mathcal{I}_\mathrm{k}} D_{\mathrm{k}+\frac{1}{2}} \frac{\Omega_{\mathrm{k}+1}^{\mathrm{n}+1} - \Omega_\mathrm{k}^{\mathrm{n}+1}}{r_{\mathrm{k}+1} - r_\mathrm{k}} + \frac{\sigma_{\mathrm{k}-\frac{1}{2}}}{\frac{3}{2}\mathcal{I}_\mathrm{k}} D_{\mathrm{k}-\frac{1}{2}} \frac{\Omega_\mathrm{k}^{\mathrm{n}+1} - \Omega_{\mathrm{k}-1}^{\mathrm{n}+1}}{r_\mathrm{k} - r_{\mathrm{k}-1}}. \tag{11}$$

By noting $[\mathrm{k}+1, \mathrm{k}] = \sigma_{\mathrm{k}+\frac{1}{2}} \frac{D_{\mathrm{k}+\frac{1}{2}}\Delta t}{r_{\mathrm{k}+1} - r_\mathrm{k}}$, and grouping similar terms together, we get:

$$\begin{aligned}
\text{For k = m (centre)} : \Omega_\mathrm{m}^\mathrm{n} &= \Omega_{\mathrm{m}-1}^{\mathrm{n}+1} \frac{[\mathrm{m}, \mathrm{m}-1]}{\frac{3}{2}\mathcal{I}_\mathrm{m}} + \Omega_\mathrm{m}^{\mathrm{n}+1}\left(1 - \frac{[\mathrm{m}, \mathrm{m}-1]}{\frac{3}{2}\mathcal{I}_\mathrm{m}}\right), \\
\text{For k = 1 (surface)} : \Omega_1^\mathrm{n} &= \Omega_2^{\mathrm{n}+1} \frac{[2, 1]}{\frac{3}{2}\mathcal{I}_1} + \Omega_1^{\mathrm{n}+1}\left(1 - \frac{[2, 1]}{\frac{3}{2}\mathcal{I}_1}\right), \\
\text{Elsewhere} : \Omega_\mathrm{k}^\mathrm{n} &= \Omega_{\mathrm{k}-1}^{\mathrm{n}+1} \frac{[\mathrm{k}, \mathrm{k}-1]}{\frac{3}{2}\mathcal{I}_\mathrm{k}} + \Omega_\mathrm{k}^{\mathrm{n}+1}\left(1 - \frac{[\mathrm{k}, \mathrm{k}-1]}{\frac{3}{2}\mathcal{I}_\mathrm{k}} - \frac{[\mathrm{k}+1, \mathrm{k}]}{\frac{3}{2}\mathcal{I}_\mathrm{k}}\right) + \Omega_{\mathrm{k}+1}^{\mathrm{n}+1} \frac{[\mathrm{k}+1, \mathrm{k}]}{\frac{3}{2}\mathcal{I}_\mathrm{k}}.
\end{aligned} \tag{12}$$

This is a system of linear equations: $\Omega^\mathrm{n} = \mathcal{M}\Omega^{\mathrm{n}+1}$, where the matrix $\mathcal{M}$ is *tridiagonal*. This kind of system can be efficiently solved with an algorithm described in Press et al. (2007).

Note that the matrix elements are only computed for the layers of the stellar interior. The envelope is not at all accounted for in the computation.

2.1.2 Resolution of the advection equation

CG: Here I give only the very minimal amount of informations I'm aware of. Georges, it would be great if you could complete with more details at some point. All the informations are in your various files, but it would be very useful to have all the informations here. Particularly, what are exactly the terms appearing in `giadv`? What are the expressions for the derivatives of these terms used in the henyey scheme? Are there some approximations done there? and so on.

The resolution of the advection of angular momentum by the meridional circulation consists in solving four differential equations:

$$\Theta = \frac{1}{3}\frac{r^2}{g}\frac{\mathrm{d}\Omega^2}{\mathrm{d}r}, \tag{13}$$

$$\mathcal{A}_\Theta = \text{Here I have troubles in identifying the terms}, \tag{14}$$

$$U_2 = \frac{P}{\overline{\rho g} C_P \overline{T}\left(\nabla_\mathrm{ad} - \nabla + \frac{\varphi}{\delta}\nabla_\mu\right)}\left[\frac{L}{M_\star}\left(E_\Omega + E_\mu\right) + \frac{C_P}{\delta}\overline{T}\partial_t\Theta\right], \tag{15}$$

$$\rho\partial_t\left(r^2\Omega\right)\big|_{M_r} = \frac{1}{5r^2}\partial_r\left(\rho r^4 \Omega U_2\right). \tag{16}$$

Expressions for E_Ω and E_μ can be found in Maeder (2009). This set of equations is iteratively solved by a relaxation method, similar to the one used for solving the structure equations (Henyey et al. 1964).

The boundary conditions are taken at the level of the first layer of the interior, and thus, the envelope is not accounted for in the computation. For example, if the first layer of the interior is radiative, the boundary condition (4) is reduced to $U_2 = 0$. If there is an external convective zone, the integration in (4) is *not* performed up to the surface, but only up to the radius of the first layer $R = r_1$.

2.2 Detailed description of the general algorithm and involved routines

The purpose of this section is to describe as precisely as possible *where* in the code are located the subroutines involved in the treatment of the transport of angular momentum, and *what* they are doing[2]. A general picture of the whole process is shown on Fig. 1. In the following text, the routines are described individually. When another subroutine is called, we advise the reader to directly jump to the description of this subroutine, and come back later to the calling point. The pdf version contains hyperlinks that allow one to directly go to the wanted section by clocking on it. At the end of this document, we also provide an index of the main variables appearing in the following sections. They are also hyperlinked (to the index).

2.2.1 `main`

The first thing made by the code in link with rotation is the reading of the binary file from the previous time-steps series. Eight important variables are read at this point: `r`, `vr`, `omegi`, `vomegi`, `CorrOmega`, `xLtotbeg`, `vsuminenv`, `dlelexprev`. At this moment, `r` is the converged radius of the previous time step, and `vr` the converged radius of the antepenultimate one. `omegi` is the estimated rotation profile of the current time step, `vomegi` the converged one of the previous time step.

After that, the number of layers `NPcoucheEff` on which the correction is applied below the surface is set, either to 200 for the main sequence, or to 50 for the advanced stages.

Once this is done, the code enters the main loop, which loops over the wanted number of time steps computed during the sequence (usually, 10, except during the very advanced stages when it is automatically put to 20 to reduce the number of structure files produced).

Here starts the main resolution loop in `main`. If a model has convergence problems, it will restart from here (see below). In case the value of `fitm` has changed, the routine `fitmshift` is called.

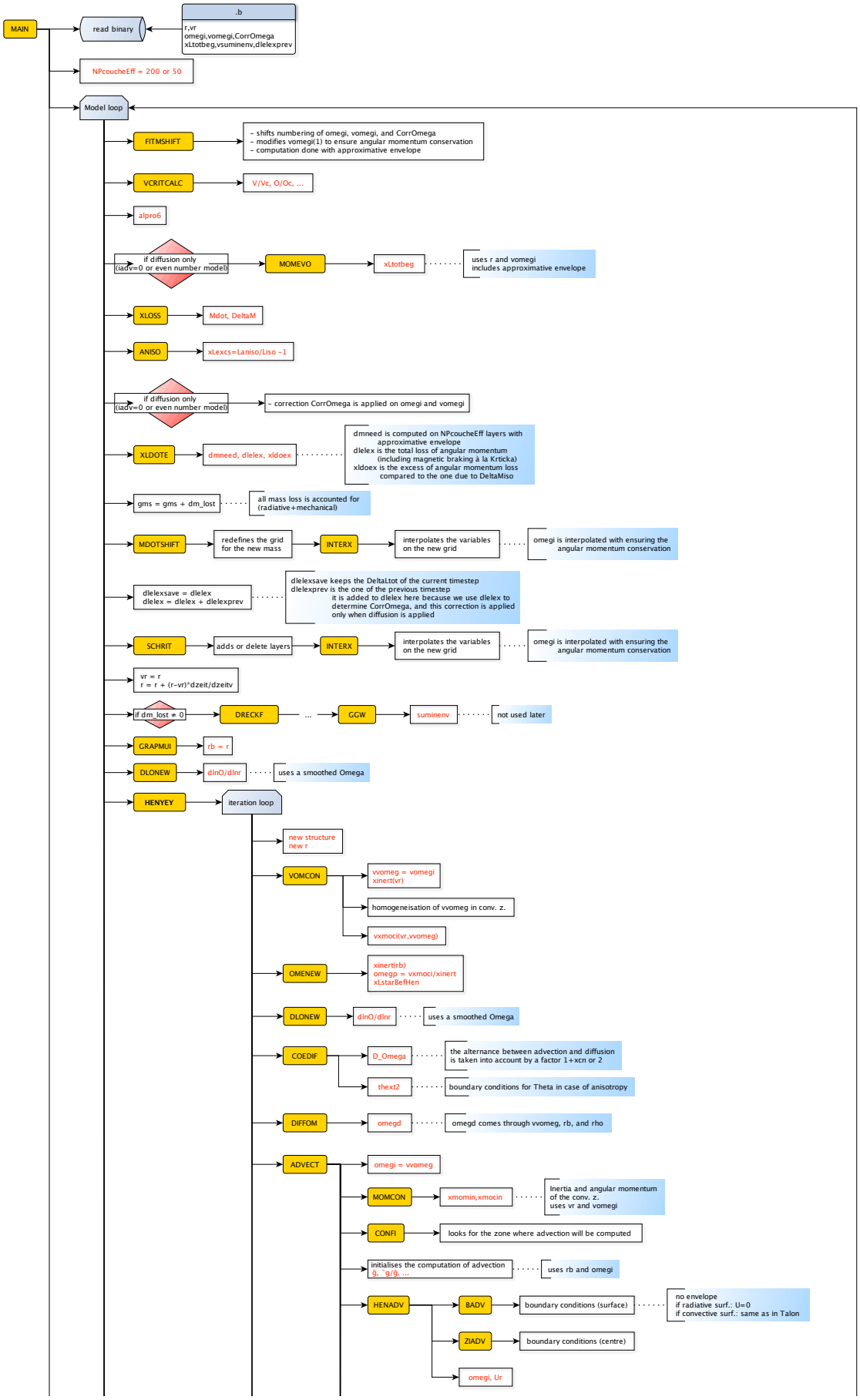
Then, `VcritCalc` is called, providing values such as the ratio $\omega = \frac{\Omega}{\Omega_{\text{crit, 1}}}$, $\omega_2 = \frac{\Omega}{\Omega_{\text{crit, 2}}}$, the equatorial velocity, the oblateness, *etc*. At this point, these are estimated values, based on the estimated temperature, luminosity, and surface angular velocity interpolated from the previous time step (non converged values).

The next step from there is the computation of the increase of the mass-loss rate due to rotation, stored in the variable `alpro6`. A detailed explanation of this computation can be found in Maeder & Meynet (2000), and some tests and correction of an error in Georgy (2010).

At this point, there is a branching depending on whether advection is applied or not during the current time step. In case `iadvec` is 1 and the time step number is even (meaning that diffusion is applied during the time step), or in case `iadvec` is 0 and `idiff` is 1 (meaning the diffusion only is applied, typically, when magnetic fields are accounted for, or during the advanced stages), `momevo` is called, and return `xLtotbeg` the total angular momentum of the star, computed with `r` and `vomegi`. Both are the converged value at the end of the previous time step, meaning that `xLtotbeg` is the total angular momentum at the end of the previous time step.

Then, `xloss` is called, computing the mass-loss rate `xmdot` as a function of stellar parameters (T_{eff}, L, surface chemical composition, *etc.*). Note that this mass-loss rate is computed on the basis of estimated parameters for the time step (non converged). This mass-loss rate is then converted into a

[2]Note that this description ignores other processes and the routines not involved in the treatment of rotation.


MAIN
read binary
.b
r,vr
omegi,vomegi,CorrOmega
xLtotbeg,vsuminenv,dlelexprev
NPcoucheEff = 200 or 50
Model loop
FITMSHIFT
- shifts numbering of omegi, vomegi, and CorrOmega
- modifies vomegi(1) to ensure angular momentum conservation
- computation done with approximative envelope
VCRITCALC
V/Vc, O/Oc, ...
alpro6
if diffusion only
(iadv=0 or even number model)
MOMEVO
xLtotbeg
uses r and vomegi
includes approximative envelope
XLOSS
Mdot, DeltaM
ANISO
xLexcs=Laniso/Liso -1
if diffusion only
(iadv=0 or even number model)
- correction CorrOmega is applied on omegi and vomegi
XLDOTE
dmneed, dlelex, xldoex
dmneed is computed on NPcoucheEff layers with
approximative envelope
dlelex is the total loss of angular momentum
(including magnetic braking à la Krticka)
xldoex is the excess of angular momentum loss
compared to the one due to DeltaMiso
gms = gms + dm_lost
all mass loss is accounted for
(radiative+mechanical)
MDOTSHIFT
redefines the grid
for the new mass
INTERX
interpolates the variables
on the new grid
omegi is interpolated with ensuring the
angular momentum conservation
dlelexsave = dlelex
dlelex = dlelex + dlelexprev
dlelexsave keeps the DeltaLtot of the current timestep
dlelexprev is the one of the previous timestep
it is added to dlelex here because we use dlelex to
determine CorrOmega, and this correction is applied
only when diffusion is applied
SCHRIT
adds or delete layers
INTERX
interpolates the variables
on the new grid
omegi is interpolated with ensuring the
angular momentum conservation
vr = r
r = r + (r-vr)*dzeit/dzeitv
if dm_lost ≠ 0
DRECKF
...
GGW
suminenv
not used later
GRAPMUI
rb = r
DLONEW
dlnO/dlnr
uses a smoothed Omega
HENYEY
iteration loop
new structure
new r
VOMCON
vvomeg = vomegi
xinert(vr)
homogeneisation of vvomeg in conv. z.
vxmoci(vr,vvomeg)
OMENEW
xinert(rb)
omegp = vxmoci/xinert
xLstarBefHen
DLONEW
dlnO/dlnr
uses a smoothed Omega
COEDIF
D_Omega
the alternance between advection and diffusion
is taken into account by a factor 1+xcn or 2
thext2
boundary conditions for Theta in case of anisotropy
DIFFOM
omegd
omegd comes through vvomeg, rb, and rho
ADVECT
omegi = vvomeg
MOMCON
xmomin,xmocin
Inertia and angular momentum
of the conv. z.
uses vr and vomegi
CONFI
looks for the zone where advection will be computed
initialises the computation of advection
g, ` g/g, ...
uses rb and omegi
HENADV
BADV
boundary conditions (surface)
no envelope
if radiative surf.: U=0
if convective surf.: same as in Talon
ZIADV
boundary conditions (centre)
omegi, Ur

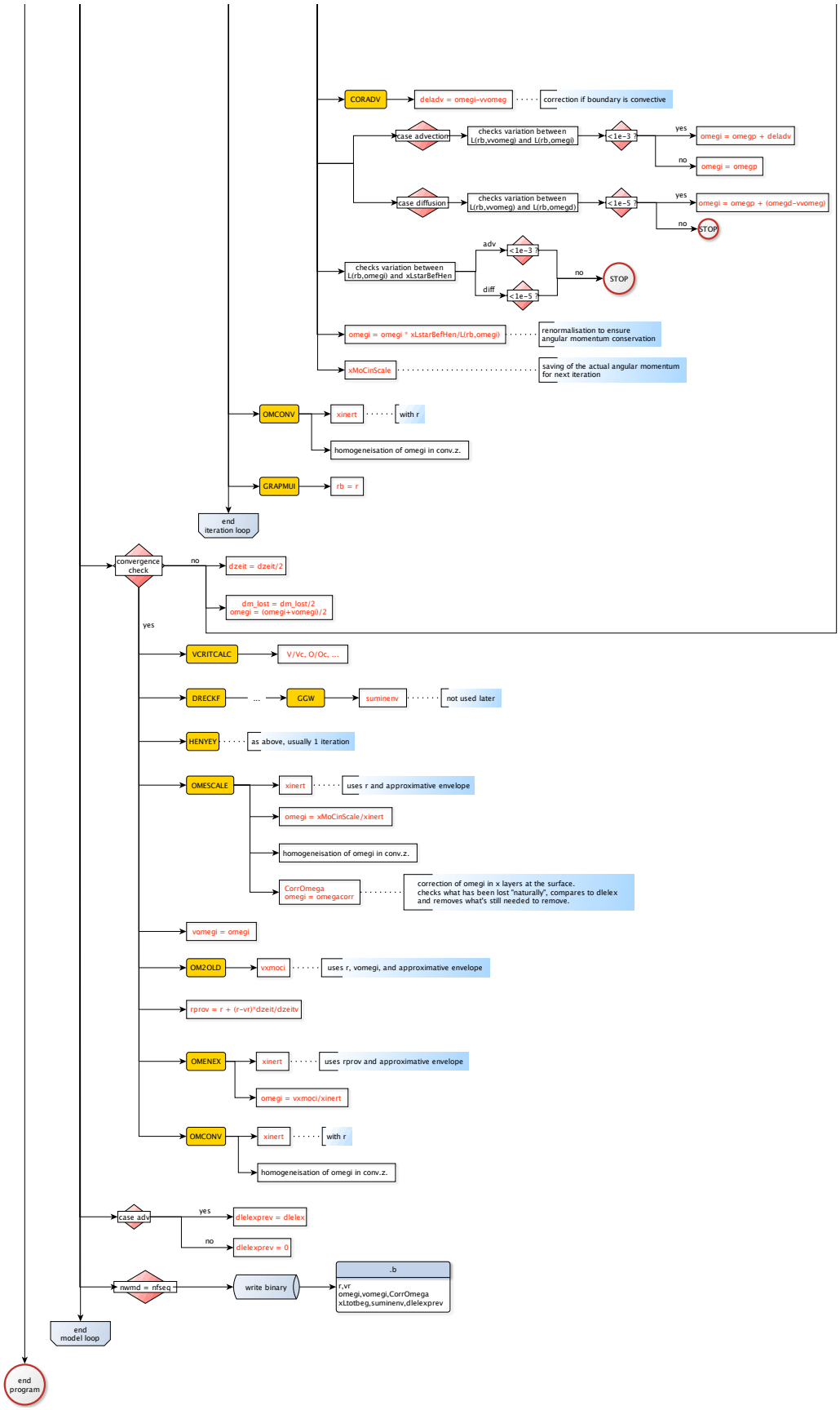


Figure 1: Flowchart of the treatment of rotation in the `Origin 2013` version of the Geneva stellar evolution code.

total mass lost during the time step, dm_lost, and is then removed from the total mass gms. Once this is done, aniso is called, returning the values bdotis and xlexcs.

The next step is again only in case iadvec is 1 and the time step number is even or in case iadvec is 0 and idiff is 1: in this situation, the rotation profiles vomegi (converged profile at the previous time step) and omegi (estimated rotation profile at the current time step) are corrected by CorrOmega[3]:

$$\begin{aligned} \texttt{vomegi} &= \texttt{vomegi} + \texttt{CorrOmega}, \\ \texttt{omegi} &= \texttt{omegi} + \texttt{CorrOmega}. \end{aligned} \tag{17}$$

A test ensures that the angular velocity remains positive there. In case it would become negative, it is reset to 10^{-20}. In principle, it never occurs, since this is already tested elsewhere (see below). At this point, we also check if the model has finished the main sequence. If this is the case, CorrOmega is completely reset to 0, since the code will switch to a fully diffusive behaviour, where the correction is applied at each time step. The number of layers on which the correction is computed NPcoucheEff is switched to 50.

Then, there is a very important call to the routine xldote, which returns the total angular momentum loss dlelex, the non-isotropic contribution to this loss xldoex, as well as an estimate of the mechanical equatorial mass loss when the star rotates critically: dmneed. Once this value is known, the total mass of the star is corrected: gms = gms + dmneed. In case the mechanical or radiative mass loss is not zero, MdotShift is called, to redefine the mesh points according to the new total mass.

To keep a correct track of the angular momentum loss, dlelex is then updated: dlelex = dlelex + dlelexprev. dlelexprev is null if the correction of the rotation profile was applied during the previous time step. Otherwise, it contains the angular momentum loss during the previous time step. At this point, dlelex contains either the angular momentum loss of the current time step, or of the two previous ones. The angular momentum loss of the current time step *only* is saved in dlelexsave.

The mesh is adapted according to some criteria (that can be set in the parameter card), through the routine schrit.

Then, an approximative model is built by interpolating the radius from the two previous time steps:

$$\begin{aligned} \texttt{r} &= \texttt{r} + \frac{\Delta t}{\Delta t_{\text{prev}}} \left(\texttt{r} - \texttt{vr}\right), \\ \texttt{vr} &= \texttt{r}. \end{aligned} \tag{18}$$

The radius r is at this point saved in the variable rb, in the routine grapmui.

The next step is the resolution of the structure equations themselves, which is done in the henyey routine. At the end of the routine, the convergence of the structure is checked. In case the model is not converged, all the converged data of the previous time step are reinitialised, and the estimated one of the current time step are recomputed as the average between the converged and the "old-estimated" ones, for example:

$$\texttt{omegi} = \frac{\texttt{vomegi} + \texttt{omegi}}{2}, \tag{19}$$

and the time step, mass loss, *etc.* are divided by 2. The code restarts the computation of the model with this reduced time step just before the call to fitmshift here above.

In case the model is correctly converged, the various critical velocities, the oblateness, and the polar and equatorial radii are recomputed on the basis of the converged model by a new call to VcritCalc. Then, henyey is called again a last time to compute the final fully converged structure, and print it in the v-file if requested.

The next major routine for the conservation of the angular momentum is the call to omescale, in charge of computing the corrections CorrOmega to be applied to the first NPcoucheEff layers

[3]Note that formally, the correction is computed from vomegi (see below), and it is thus only an approximation to apply it directly to omegi. It provides however a better estimate for the angular momentum content of the star.

below the surface. Once this is done, the converged Ω-profile is saved in `vomegi`:

$$\texttt{vomegi} = \texttt{omegi}. \tag{20}$$

The momentum of inertia of each shell is computed in `om2old`, and stored in the variable `vxmoci`.

What remains in `main` is the computation of an estimate of the rotation profile at the next time step. First, the radial structure of the next time step is estimated:

$$\texttt{rprov} = \texttt{r} + \frac{\Delta t}{\Delta t_{\text{prev}}}(\texttt{r} - \texttt{vr}). \tag{21}$$

Then, a new rotation profile `omegi` is computed assuming the local conservation of angular momentum between the radial structure `r` and `rprov` in the routine `omenex`. This new Ω-profile is then homogenised in the convective zones with another call to `omconv`.

The last step about the treatment of the transport of angular momentum in `main` is the saving of `dlelexprev`. If the current model number is odd, meaning that the advection was applied, then `dlelexprev` is set to `dlelex` (the angular momentum lost during the current time step is saved). In the other case, `dlelexprev` is reinitialised to 0.

Finally, if the current model number is the last of a series, all the relevant data about the rotation and angular momentum transport are saved in the binary file: `r`, `vr`, `omegi`, `vomegi`, `CorrOmega`, `xLtotbeg`, `vsuminenv`, abd `dlelexprev`. In case a new model has to be computed, the whole process is repeated, starting again just before the call to `fitmshift` here above. Otherwise, the execution stops.

2.2.2 `fitmshift`

This routine aims to manage the mass meshing when `fitm` is changed in the course of a computation. In the 2013 version, we only care about the case of a decrease of `fitm`. In this case, the following steps are performed:

1. The number of layers that have to be removed, n_0, is computed.

2. The angular momentum contained in the removed layers is computed, accounting for the envelope in a simple way (assuming all of its mass is concentrated at the same radius as the first layer of the interior):

$$\mathcal{L}_{\text{old}} = \mathcal{I}_{\text{env, old}}(\Omega_{1,\,\text{old}} + \Delta\Omega_{1,\,\text{old}}) \sum_{i=0}^{n_0+1} \mathcal{I}_{i,\,\text{old}}(\Omega_{i,\,\text{old}} + \Delta\Omega_{i,\,\text{old}}), \tag{22}$$

   where is indices "old" refers to the old numbering of the layers (see below). Note here the following points:

   - In the Geneva code, the momentum of inertia of each layer of the interior, except the first one and last one, is given by:

$$\mathcal{I}_i = \frac{2}{3}\frac{M_{i-1} - M_{i+1}}{2} r_i^2. \tag{23}$$

     For the surface, it is:

$$\mathcal{I}_1 = \frac{2}{3}\frac{M_1 - M_2}{2} r_1^2, \tag{24}$$

     and for the centre:

$$\mathcal{I}_m = \frac{2}{5}\frac{M_{m-1}}{2} r_m^2. \tag{25}$$

     For the envelope, we consider here that all the mass is concentrated on the very top of the first layer:

$$\mathcal{I}_{\text{env}} = \frac{2}{3}(1 - \text{fitm}) M_{\text{tot}} r_1^2, \tag{26}$$

and that it is rotating at the same velocity as the first layer of the interior.
Due to the way the mass of a layer is computed (involving the $n-1$the layer), we need to consider all the layers up to the n_0+1th one in the sum of Eq. 22.

- At this point, `omegi` contains the non corrected value of the angular velocity. We thus need to account for the correction `CorrOmega` in order to get the correct angular momentum.

3. The numbering of the layers is shifted by n_0: $n_0 \to 1$, $n_0+1 \to 2$, *etc.* The mass coordinate of the first layer is set to `fitm`.

4. We adjust the angular velocity of the first layer and the envelope by imposing:

$$\mathcal{L}_{\text{old}} = \mathcal{I}_{\text{env, new}}\Omega_{1,\text{ new}} + \sum_{i=1}^{2}\mathcal{I}_{i,\text{ new}}\Omega_{i,\text{ new}}. \tag{27}$$

This leads to the following relation for $\Omega_{1,\text{ new}}$:

$$\Omega_{1,\text{ new}} = \frac{\mathcal{L}_{\text{old}} - \mathcal{I}_2\Omega_{2,\text{ new}}}{\mathcal{I}_{1,\text{ new}} + \mathcal{I}_{\text{env, new}}}. \tag{28}$$

This modification is absorbed in the correction of the angular velocity:

$$\Delta\Omega_{1,\text{new}} = \Omega_{1,\text{ new}} - \Omega_{n_0,\text{ old}}. \tag{29}$$

2.2.3 `VcritCalc`

This routine computes several quantities linked with the surface rotation, such as the oblateness, the ratios $\Omega/\Omega_{\text{crit, 1}}$, $\Omega/\Omega_{\text{crit, 2}}$, $v/v_{\text{crit, 1}}$, and $v/v_{\text{crit, 2}}$, the polar and equatorial radii, the equatorial linear velocity. The computation is done starting from the normalised value of the stellar surface, used to interpolate the oblateness in the file `ftfp.dat`. The velocity used here is the current value stored in `omegi`.

The inverse of the oblateness `f` is computed as well, and used again later. The details of the computations can be found in Ekström et al. (2008) and Maeder (2009).

2.2.4 `momevo`

`momevo` computes the total angular momentum of the star. It takes four input parameters, a radii profile, an angular velocity profile, the correction applied to the angular velocity, and a logical. It returns the total angular momentum of the star, computed as in `fitmshift` (but obviously through all the layers). If the logical is `True`, then the envelope (again computed as in `fitmshift`) is *not* included in the total. Otherwise, it is included.

2.2.5 `xloss`

This routine is in charge of the computation of the mass-loss rate at different phase of the evolution of the star. A complete description of the different recipes used here can be found in Georgy (2010).

2.2.6 `aniso`

In this routine, the shape of the surface of the star is computed as a function of the current `f` parameter of the star. This allows one to compute for each colatitude on the star the local temperature, gravity, and fraction of the mass flux emerging at this point. It is then rescaled to the total value of the mass loss `dm_lost`. Then, the angular momentum flux is computed at each colatitude and integrated over the surface. A complete description of this procedure can be found in Maeder (2002), Georgy (2010), and Georgy et al. (2011).

Finally, the routine returns the excess/lack of angular momentum loss with respect to the isotropic case `xlexcs`, and the value of the angular momentum lost if we assume it is isotropic `bdotis`.

2.2.7 `xldote`

The routine `xldote` computes several quantities linked with the angular momentum loss, and provides an estimate of the equatorial "mechanical" mass loss needed in order to keep the star at the critical velocity when it reaches it.

Estimate of the mechanical mass loss. The account for the mechanical mass loss is controlled by the parameter `rapcrilim`. If it is set to 0, then the computation of this mechanical mass loss is jumped. Otherwise, `rapcrilim` defines the maximal ratio $\texttt{rapcrilim} = \omega_{\rm lim} = \frac{\Omega_{\rm lim}}{\Omega_{\rm crit}}$ authorised for the star. It is by default set to 0.99, meaning that the code will compute the mechanical mass loss needed to keep the star at this fraction of the critical velocity. First of all, the maximal authorised angular velocity is computed (in the equations shown in this section, when a quantity is first define, we also write `in teletype font` the name of the variable in the code. We then use only the mathematical symbol to save space[4].):

$$\Omega_{\rm lim} = \texttt{omlim} = \omega_{\rm lim}\frac{\Omega_1}{\omega}. \tag{30}$$

The total angular momentum loss during the current time step is also computed (accounting for the effects of the anisotropic stellar wind if needed):

$$\Delta\mathcal{L} = \texttt{dlelex} = -\texttt{bdotis}(1+\texttt{xlexcs}). \tag{31}$$

By assuming that the loss of angular momentum will affect `NPcoucheEff` (noted here $N_{\rm corr}$) layers by modifying their angular velocity as:

$$\Omega^{\rm new}_{i,\, i\in[1,N_{\rm corr}]} = \Omega_i\,(1+q)\,, \tag{32}$$

we can show by imposing the total angular momentum conservations that q is (Georgy 2010; Georgy et al. 2013):

$$q = \frac{\mathcal{L}_{\rm e,\,ini}\frac{\Delta M_{\rm rad}}{m_{\rm e,\,ini}} - \Delta\mathcal{L}}{\mathcal{L}_{\rm tot,\,ini} - \mathcal{L}_{\rm e,\,ini}\frac{\Delta M_{\rm rad}}{m_{\rm e,\,ini}}}, \tag{33}$$

where

$\mathcal{L}_{\rm tot,\,ini} = \mathcal{L}_{\rm e,\,ini} + \sum_{i=1}^{N_{\rm corr}}\mathcal{L}_{i,\,\rm ini}$ (where the subscript "e" means "envelope"),

$\mathcal{L}_{\rm tot,\,fin} = \mathcal{L}_{\rm e,\,ini}\,(1+q)\left(1-\frac{\Delta M_{\rm rad}}{m_{\rm e,\,ini}}\right) + (1+q)\sum_{i=1}^{N_{\rm corr}}\mathcal{L}_{i,\,\rm ini}$,

$\Delta M_{\rm rad}$ is the total mass loss during the time step,

$m_{\rm e,\,ini}$ the initial mass of the envelope.

The estimated surface angular velocity becomes thus:

$$\Omega_1^{\rm new} = \texttt{newomega} = \Omega_1(1+q). \tag{34}$$

Then, this value is compared to $\Omega_{\rm lim}$. In case $\Omega_1^{\rm new} > \Omega_{\rm lim}$, the mechanical mass loss `dmneed` is estimated as follows (Georgy 2010; Georgy et al. 2013):

$$\Delta M_{\rm mech} = \texttt{dmneed} = \frac{\mathcal{L}_{\rm tot,\,ini}\left(1-\frac{\Omega_{\rm lim}}{\Omega_{1,\,\rm ini}}\right) + \mathcal{L}_{\rm e,\,ini}\frac{\Delta M_{\rm rad}}{m_{\rm e,\,ini}}\frac{\Omega_{\rm lim}}{\Omega_{1,\,\rm ini}} - \Delta\mathcal{L}}{\Omega_{1,\,\rm ini}r_{\rm eq}^2 - \frac{\mathcal{L}_{\rm e,\,ini}}{m_{\rm e,\,ini}}\frac{\Omega_{\rm max}}{\Omega_{1,\,\rm ini}}}, \tag{35}$$

with $r_{\rm eq}$ the equatorial radius of the star, which is computed in `aniso`. We assume here that this mass is lost at the equator in a circumstellar disc. In case ω is significantly higher than $\omega_{\rm lim}$ at the beginning of the time step, this mass loss is multiplied by a factor higher than 1 (depending how far from $\omega_{\rm lim}$ we are). The corresponding angular momentum loss `dLmeca` is thus:

$$\Delta\mathcal{L}_{\rm mech} = \texttt{dLmeca} = \Delta M_{\rm mech}\Omega_{\rm surf}r_{\rm eq}^2. \tag{36}$$

[4]Moreover, the following notations are used: Ω_i is the angular velocity of the ith layer, ω is the ratio $\frac{\Omega_1}{\Omega_{\rm crit}}$ where $\Omega_{\rm crit}$ is the critical velocity.

Total angular momentum loss. In a next step, the routine `dLmagcalc` is called, returning the angular momentum loss due to magnetic braking `dLmag` (by construction of the formula used, `dLmag` contains the contribution of both the usual isotropic loss due to radiative winds and the contribution from the the magnetic braking). The total angular momentum loss during the time step is recomputed as:

$$\texttt{dlelex} = \underbrace{\texttt{dLmag}}_{\text{isotropic winds + magnetic braking}} - \underbrace{\texttt{bdotis} \cdot \texttt{xlexcs}}_{\text{anisotropic winds contribution}} + \underbrace{\texttt{dLmeca}}_{\text{mechanical mass loss}}, \tag{37}$$

and the angular momentum in excess compared to the isotropic case (used as boundary condition in the advection computation, see Maeder 2002):

$$\dot{\mathcal{L}}_{\text{excess}} = \texttt{xldoex} = \frac{\overbrace{\texttt{bdotis} * \texttt{xlexcs}}^{\text{anisotropic contribution only}}}{\Delta t}. \tag{38}$$

Note that this is correct in case of anisotropic radiative winds, but is not consistent in case a mechanical mass loss or a magnetic braking is accounted for.

2.2.8 `dLmagcalc`

This routine takes as input the isotropic angular momentum loss during the time step, and returns its value when a given magnetic breaking is applied (or return the same value in case magnetic braking is not accounted for). The formalism is the same as in Meynet et al. (2011) and Ud-Doula et al. (2009):

$$\dot{\mathcal{L}}_{\text{mag}} = \frac{2}{3}\dot{M}\Omega_1 R_\star^2 \left[0.29 + (\eta_\star + 0.25)^{\frac{1}{4}}\right]^2, \tag{39}$$

where $R_\star$ is the stellar radius, and $\eta_\star = \frac{B_{\text{eq}}^2 R_\star^2}{\dot{M} v_\infty}$. B_{eq} is the magnetic field at the equator at the stellar surface, and v_∞ is the terminal wind velocity. For estimating this velocity, we use the parametrisation from Kudritzki & Puls (2000):

$$v_\infty = C\left(T_{\text{eff}}\right) v_{\text{esc}}, \tag{40}$$

with

$$C\left(T_{\text{eff}}\right) = \begin{cases} 2.65 & T_{\text{eff}} > 21000\,[\text{K}] \\ 1.4 & 10000 < T_{\text{eff}} \leq 21000\,[\text{K}] \\ 1 & T_{\text{eff}} \leq 10000\,[\text{K}] \end{cases}. \tag{41}$$

2.2.9 `MdotShift`

This routine is used to redefine the mass mesh `q` inside the star once the mass has been adjusted due to mass loss. It also interpolates the values at the centre and the surface. In more details:

- The position of the new `fitm` is found in the old grid. The first layer is redefine at this position, and all the quantities are interpolated at this point. Concerning `vomegi` and `omegi`, see the note below.
- In case the new `fitm` falls *not* between the first and second layer of the old meshing, the required layers are removed.
- The layer number $m - 1$ is redefined and the values interpolated on its new position.
- The mass coordinate `q` (which depends on the total mass) is redefined everywhere, in order to keep the *real mass* position constant.

Important note: In case of very strong mass loss, more than one layer might have to be removed from the interior. In case a very strong gradient of Ω is present in this region, the interpolation process makes the surface rotate suddenly very fast (usually above the critical limits), such that the code crashes. This is due to the fact that the surface rotates implicitly at the same angular velocity as the first layer of the interior. To improve this situation, the following is now done in this routine. By design, `MdotShift` removes a fraction `fitm` of the total mass loss in the interior (in fact, it repositions $q(1)$ inside the star, and removes the possible layers of the previous models that are above this value), and a fraction 1-`fitm` in the envelope. So, we assume that:

1. the velocity of the envelope should not change: it remains at the same velocity $\Omega_{1,\,\text{prev}}$ as before, since the mass removed from the envelope carries out its own angular momentum, and thus does not affect the surface angular velocity.

2. the angular velocity of the first layer inside the star takes an interpolated value $\Omega_{1,\,\text{interp}}$. To stay in the design of the code, we would require that the angular velocity of the envelope $\Omega_{1,\,\text{prev}}$, and the velocity of the first layer $\Omega_{1,\,\text{interp}}$ are the same. However, it has no reason to be true.

So a new surface (and first layer) velocity is computed, imposing angular momentum conservation and equalising the angular velocity of both zones. Technically, it is done as follows. The momentum of inertia and the angular momentum of the envelope and first layer are[5]:

$$\begin{aligned}
\mathcal{I}_\text{e} &= \frac{2}{3}M_\text{e}r_1^2 = \frac{2}{3}(1-\texttt{fitm})M_\text{tot}r_1^2 = \frac{2}{3}\exp(q_1)M_\text{tot}r_1^2,\\
\mathcal{L}_\text{e} &= \mathcal{I}_\text{e}\Omega_{1,\,\text{prev}},\\
\mathcal{I}_1 &= \frac{2}{3}M_1r_1^2 = \frac{2}{3}(\exp(q_2)-\exp(q_1))M_\text{tot}r_1^2,\\
\mathcal{L}_1 &= \mathcal{L}_1\Omega_{1,\,\text{interp}}.
\end{aligned} \tag{42}$$

The new surface angular velocity is then reset to:

$$\Omega_{1,\,\text{new}} = \frac{\mathcal{L}_\text{e}+\mathcal{L}_1}{\mathcal{I}_\text{e}+\mathcal{I}_1}. \tag{43}$$

2.2.10 `schrit`

This routine parses the mesh, and ensures that the number of radial zones is enough according to some criteria (some of them can be set in the parameter card). In case there is too much zones at some places, they are removed. In case the discretisation is too coarse, layers are added and the other ones are renumbered. This is done with ensuring the conservation of the total angular momentum.

2.2.11 `grapmui`

`grapmui` computes the mean molecular weight gradient ∇_μ. In case `idialu` is set to 1, this gradient is smoothed, producing cleaner profiles to be used for the computation of the advection.

Importantly for the computation of the angular momentum, the radius at the previous iteration `rb` is saved in this routine as well.

2.2.12 `dlonew`

This routine computes the value of $\frac{\mathrm{d}\ln(\Omega)}{\mathrm{d}\ln(r)}$. The values of the current iteration `omegi` and `r` are used. If `idialo` is set to 1, it computes a smoothed profile of Ω and $\frac{\mathrm{d}\ln(\Omega)}{\mathrm{d}\ln(r)}$.

[5]These momentums are computed in the same way as in `fitmshift`.

2.2.13 **henyey**

henyey is the main routine used to solve the structure equations. It uses a relaxation method, proposed by Henyey et al. (1964). The purpose of this document is not to describe how this is done, and the details can be found in any textbook about stellar evolution (*e.g.* Kippenhahn & Weigert 1990; Maeder 2009). By design, the Geneva code does not solve simultaneously the structure equations, the change in the chemical composition (due to nuclear burning and transport processes) and the transport of angular momentum. It proceeds step by step, looping as follows:

1. one iteration for solving the structure is done;
2. the transport and burning of chemical species is computed, as well as the transport of angular momentum;
3. the convergence of the structure is tested. In case it has not converged, the loop is reiterated.

The following gives a detailed explanation about the transport of angular momentum resolution. Immediately after the first part of the routine, we thus have a new structure, particularly new radii r (and also, *e.g.* new convective zones, *etc.*). At this stage, the routine vomcon is called, homogenising the angular velocity in the convective zones. Moreover, it returns the angular momentum of each layers vxmoci.

Once this is done, the code computes the new rotation profile on the structure of the previous iteration (rb) by applying the local conservation of the angular momentum. This is done by the subroutine omenew, which moreover returns the total angular momentum *of the interior* xLstarbefHen. The derivative $\mathrm{d}\ln(\Omega)/\mathrm{d}\ln r$ is recomputed (with omegi and r) in dlonew.

The various diffusion coefficients are then computed in coedif. With them, the diffusion of the angular momentum is computed in diffom, which returns a new rotation profile omegd. In a next step, the routine advect is called. Finally, the Ω-profile is homogenised in the convective zones in the routine omconv. Then, grapmui is called again, and stores the radial structure r in rb for the next structure iteration. The whole process starts again from the beginning of henyey, until the structure is converged or the maximum number of iterations is reached.

2.2.14 **vomcon**

This routine makes three different operations on the vomegi profile (let us recall here that vomegi is here the converged Ω-profile at the end of the previous time step):

- vomegi is saved in the variable vvomeg.
- vvomeg is homogenised in the *current* convective zones. To proceed, the code computes in each convective zone the total momentum of inertia $\mathcal{I}_{\mathrm{CZ}}$ and angular momentum $\mathcal{L}_{\mathrm{CZ}}$, and redefines Ω in these regions as $\Omega_{\mathrm{CZ}} = \frac{\mathcal{L}_{\mathrm{CZ}}}{\mathcal{I}_{\mathrm{CZ}}}$.
- The angular momentum of each layer, computed with the radii vr (*i.e.* the converged radius of the previous time step) and the angular velocities vomegi (*i.e.* the converged Ω-profile of the previous time step) is stored in vxmoci.

2.2.15 **omenew**

This routine computes the new Ω profile by applying the local conservation of the angular momentum. The starting point is vxmoci computed in vomcon. This angular momentum is divided by the momentum of inertia computed with the radii of the *previous* iteration in henyey (or the estimated radii rb computed in main during the first iteration). The new Ω-profile is thus computed as:

$$\mathtt{omegp} = \frac{\mathtt{vxmoci}\,(\mathtt{vr}, \mathtt{vomegi})}{\mathcal{I}\,(\mathtt{rb})}. \tag{44}$$

To summarise, we thus have:

$$\left.\begin{array}{l}\overbrace{\begin{array}{ll}\text{converged radius} & \texttt{vr} \\ \text{converged } \Omega & \texttt{vomegi}\end{array}\Big\}\texttt{vxmoci}}^{\text{previous time step}} \\ \\ \underbrace{\text{previous iteration radius} \quad \texttt{rb}\big\}\,\mathcal{I}(\texttt{rb})}_{\text{current time step}}\end{array}\right\} \Rightarrow \texttt{omegp} \text{ (local angular mom. cons.)}$$

At the end of the routine, the total angular momentum of the interior of the model (*i.e.* without the envelope) is stored in the variable `xLstarbefHen`. It will be used later on to check the conservation of angular momentum during the advection/diffusion computation.

2.2.16 `coedif`

This routine computes the various diffusion coefficient linked with the transport of chemical species and angular momentum. We do not aim in this document at providing a detailed description of what is done here (see *e.g.* Maeder 1997; Talon & Zahn 1997; Zahn 1992; Maeder 2003; Mathis et al. 2004, or the code documentation available through svn.). The choice of the diffusion coefficient is done through the parameter `icoeff` of the parameter card, and the inclusion of the so-called "Tayler-Spruit" dynamo (Spruit 2002) in its revised version (Maeder & Meynet 2003, 2004, 2005) is controlled by the parameter `imagn` of the parameter card. The diffusion coefficients are stored in the variables `D_chim` for the chemical species, and `D_Omega` for the angular momentum.

An important point to be noted here is that, because the design of the Geneva code imposes advection and diffusion being applied separately each two time steps, the diffusion coefficient is here multiplied by a factor of 2 (or 1 + xcn at the first time step of a series).

In this routine is also computed the boundary condition for the first equation concerning advection (13). According to Maeder (2002), the boundary condition is:

$$\partial_r \Omega = \frac{3}{8\pi} \frac{\dot{\mathcal{L}}_{\text{excess}}}{\rho D_{\text{shear}} r^4}. \tag{45}$$

Here we used the shear diffusion coefficient from Maeder (1997):

$$D_{\text{shear}} = f_{\text{energ}} \frac{H_P}{g\delta} \frac{K}{\frac{\varphi}{\delta}\nabla_\mu + \nabla_{\text{ad}} - \nabla_{\text{rad}}} \left(\frac{9\pi}{32} r \partial_r \Omega\right)^2 \equiv A(\partial_r \Omega)^2. \tag{46}$$

Putting this together, we get:

$$\partial_r \Omega = \left(\frac{3}{8\pi} \frac{\dot{\mathcal{L}}_{\text{excess}}}{\rho r^4 A}\right)^{\frac{1}{3}}. \tag{47}$$

Using the definition of θ (eq. 13), we finally get:

$$\texttt{thext2} = \Theta = \frac{2}{3}\frac{r^2\Omega}{g}\partial_r\Omega = \frac{2}{3}\frac{r^2\Omega}{g}\left(\frac{3}{8\pi}\frac{\dot{\mathcal{L}}_{\text{excess}}}{\rho r^4 A}\right)^{\frac{1}{3}}. \tag{48}$$

Here, $\dot{\mathcal{L}}_{\text{excess}}$ is the excess of angular momentum with respect to the isotropic case, and is stored in `xldoex` from the routine `xldote`. Note also that formally, this boundary condition is inconsistent for another choice of D_{shear} as the on from Maeder (1997).

2.2.17 `diffom`

The routine `diffom` computes the diffusion of the angular momentum, with the diffusion coefficient computed in `coedif`. The numerical method is an implicit finite differences scheme (see Sect. 2.1.1). The time step is divided in a number of sub-step, according to the Courant condition.

The matrix inversion is done by the mean of an LU decomposition, which is very efficient thanks to the tridiagonal property of the system (see Press et al. 2007). Schematically, we have here:

$$\left.\begin{array}{l} \overbrace{\begin{array}{l}\text{radius} \qquad\qquad \texttt{rb} \\ \text{diffusion coefficients}\end{array}}^{\text{previous iteration}} \\ \\ \underbrace{\text{converged } \Omega \quad \texttt{vvomeg}}_{\text{previous time step}} \end{array}\right\} \Rightarrow \texttt{omegd}$$

2.2.18 `advect`

This routine contains quite a lot of different cases. Let us examine them one by one.

In case `iadvec` is 1: the advection is computed. This needs several pre-computations. First, `omegi` is re-initialised to `vvomeg`. The momentum of inertia `xmomin` and the angular momentum `xmocin` of each convective zones are computed in `momcon`. Then, the configuration of the convective zones inside the star is checked, and the momentum of inertia and angular momentum of the relevant ones are provided in `xbint`, `xcint`, `xbext`, and `xcext` by a call to the routine `confi`.

In case the advection needs to be computed (*i.e.* `inoadv` has not been set to 1 in `confi`), the code then calls a set of routines initialising various quantities useful for the advection, such as $\frac{\tilde{g}}{g}$, *etc.* They are computed using the structure of the previous iteration `rb`, and the Ω-profile stored in `omegi`.

Once this is done, the advection part of the angular momentum transport equation (16) is solved using a relaxation scheme (see Section 2.1.2) by calling the routine `henadv`. At the end of the process, we get `omegi` the rotation profile solution of the equation. This solution is then applied in the whole star (including the zones where advection was not computed) in the routine `coradv`, which returns the *difference* between the final solution of the advection `omegi` and the initial profile `vvomeg`, stored in the variable `deladv`.

Then, the code behaves again differently depending on the parity of the time step number:

- *If the time step is odd (advection applied):* The conservation of the angular momentum during the computation of advection is checked: the total angular momentum (interior only) computed with the structure `rb` and rotation profile `vvomeg` and computed with `rb` and `omegi` are compared. In case it varies more than 10^{-3} (relatively), or the maximum number of iterations was reached when computing the advection in `henadv`, then the computation of advection is rejected[6]. Otherwise, it is accepted. At this stage, the *actual* rotation profile is corrected as follows:

  $$\texttt{omegi} = \begin{cases} \texttt{omegp} + \texttt{deladv} & \text{if accepted} \\ \texttt{omegp} & \text{if rejected} \end{cases}.$$

  (Note that the meaning of `omegi` changes here: before, it was the actual solution of the advection equation starting from the the initial profile `vvomeg`, and from now on, it is the profile where the local conservation of angular momentum was applied, and corrected for the effects of advection through `deladv`.)

  Then, the total angular momentum of the interior is again recomputed with the new `omegi` and `rb`, and compared with the angular momentum content after the local conservation of angular momentum, computed with `rb` and `omegp`. In case the relative variation is bigger than 10^{-3}, the execution of the code stops.

[6]During the very first models at the beginning of the Main Sequence, where the Ω-profile has to adjust quite a lot, the computation of advection is accepted, even if the variation is bigger than 10^{-3}. This is possible only while the central H fraction is in the range $[X_{\text{ini}} - 5 \cdot 10^{-3}, X_{\text{ini}}]$.

Schematically, these steps are summarised as:

$$\left.\begin{array}{c}\texttt{vvomeg}\\ \texttt{rb}\\ \mathcal{L}_{\text{bef. }\texttt{henadv}}\end{array}\right\}\xrightarrow{\texttt{henadv}}\left\{\begin{array}{c}\Omega_{\text{advected}}\\ \texttt{rb}\\ \mathcal{L}_{\text{after }\texttt{henadv}}\end{array}\right\}\rightarrow\begin{array}{c}\texttt{deladv}=\\ \Omega_{\text{advected}}-\texttt{vvomeg}\end{array}\left\{\begin{array}{c}\underbrace{\texttt{omegi}=\texttt{omegp}+\texttt{deladv}}_{\text{if accepted}}\\ \texttt{rb}\\ \mathcal{L}_{\text{final}}\\ \\ \underbrace{\texttt{omegi}=\texttt{omegp}}_{\text{if rejected}}\\ \texttt{rb}\\ \mathcal{L}_{\text{final}}\end{array}\right\}$$

$$\mathcal{L}_{\text{final}}\text{ compared with }\texttt{xLstarbefHen}\left\{\begin{array}{c}\text{accepted}\\ \\ \text{rejected}\rightarrow\text{stop}\end{array}\right.$$

- *If the time step is even (diffusion applied):* The conservation of the angular momentum during the computation of diffusion is checked: the total angular momentum (interior only) computed with the structure `rb` and rotation profile `vvomeg` and computed with `rb` and `omegd` are compared. In case it varies more than 10^{-5} (relatively) then the computation of diffusion is rejected and the execution stopped. Otherwise, it is accepted. At this stage, the *actual* rotation profile is corrected as follows:

$$\texttt{omegi}=\texttt{omegp}+\texttt{omegd}-\texttt{vvomeg}.$$

  Then, the total angular momentum of the interior is again recomputed with the new `omegi` and `rb`, and compared with the angular momentum content after the local conservation of angular momentum, computed with `rb` and `omegp`. In case the relative variation is bigger than 10^{-5}, the execution of the code stops.

  Schematically, these steps are summarised as:

$$\left.\begin{array}{c}\texttt{vvomeg}\\ \texttt{rb}\\ \mathcal{L}_{\text{bef. }\texttt{diffom}}\end{array}\right\}\xrightarrow{\texttt{diffom}}\left\{\begin{array}{c}\texttt{omegd}\\ \texttt{rb}\\ \mathcal{L}_{\text{after }\texttt{diffom}}\end{array}\right\}\left\{\begin{array}{c}\underbrace{\texttt{omegi}=\texttt{omegp}+\texttt{omegd}-\texttt{vvomeg}}_{\text{if accepted}}\\ \texttt{rb}\\ \mathcal{L}_{\text{final}}\\ \\ \text{stop if rejected}\end{array}\right\}$$

$$\mathcal{L}_{\text{final}}\text{ compared with }\texttt{xLstarbefHen}\left\{\begin{array}{c}\text{accepted}\\ \\ \text{rejected}\rightarrow\text{stop}\end{array}\right.$$

In both cases, the Ω-profile are rescaled in order to ensure the conservation of angular momentum:

$$\texttt{omegi}=\texttt{omegi}\frac{\texttt{xLstarbefHen}}{\mathcal{L}_{\text{final}}}. \tag{49}$$

and the angular momentum of each layers of the star, computed with `rb` and the rescaled `omegi` are stored in the variable `xMoCinScale`. This rescaling is mandatory due to the design of the code. Indeed, even with an advection/diffusion scheme that would perfectly conserve angular momentum, since the input Ω-profile is `vvomeg` and *not* `omegp`, there is no reason why the step from `omegp` to `omegi` described above would conserve it.

In case `iadvec` is 0: Advection is not computed, and the code directly jumps to the part described in the second item above (case where diffusion is applied).

2.2.19 `momcon`

For each convective zone inside the star, the total momentum of inertia is computed and stored in `xmomin`, and the total angular momentum is computed and stored in `xmocin`. They are both computed with the converged values of the radii and the angular velocities at the previous time step `vr` and `vomegi`. These quantities are computed as usual elsewhere in the code. Note that the envelope is *not* accounted for in case the surface is convective.

2.2.20 `confi`

`confi` stores in `xbint`, `xcint`, `xbext`, and `xcext` the values of the momentum of inertia and of the angular momentum of the relevant convective zones, according to a convective configuration computed in `rechzco` (not described here). The following cases are considered:

0. The star is entirely convective. In this case, advection is not considered, and `inoadv` is set to 1.
1. The star is entirely radiative. The advection is computed from the centre to the surface.
2. The star has a radiative centre and surface, but has some intermediate convective zones. In this case, advection is computed between the surface and the first convective zone encountered going toward the centre.
3. The star is convective at the centre and surface, and has other convective zones in between. In this case, advection is applied in the first radiative zone below the surface.
4. The star is radiative at the centre and convective at the surface. In this case, advection is applied in the first radiative zone below the surface (possibly going up to the centre).
5. The star is convective at the centre and radiative at the surface. Advection is considered from the surface to the first convective zone encountered going toward the centre.

2.2.21 `henadv`

This routine aims to solve the advective part of the angular momentum transport equation (16), by mean of a relaxation scheme similar to the one used in `henyey` to solve the structure equation. It starts with the rotation profile `vvomeg`, and iteratively solves the equation up to the final solution, which is then stored in `omegi`.

The boundary conditions (4, 5) are defined in `badv` for the surface, and `ziadv` for the centre. In case `xldoex` is not zero, the boundary condition for $\partial_r \Omega$ (3) is expressed as a function of Θ:

$$\Theta_1 = \texttt{thext2}, \tag{50}$$

where `thext2` comes from `coedif`. The matrix elements for the interior are computed in `giadv`.

Schematically, we have:

$$\left.\overbrace{\begin{array}{ll}\text{radius} & \texttt{rb} \\ \Omega\text{-profile} & \texttt{omegi}\end{array}}^{\text{previous iteration}}\right\} \Rightarrow \text{converged } \texttt{omegi}$$

2.2.22 `badv`

This routine computes the matrix element called "`b1`" and its derivatives, that encodes the boundary condition (4). If there is an external convective boundary, the boundary condition is rewritten as follows. First, we use relation (9) to get:

$$\frac{3}{8\pi}\partial_t\left(\Omega\mathcal{I}_{\text{ext}}\right) = -\frac{1}{5}r^4\rho\Omega U_2. \tag{51}$$

We then assume that only Ω is varying during the time step (the background structure is fixed), and that the angular velocity of the previous time step is equal to $\frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}$ (in order to account for the possible change of the convective zones). We have thus:

$$\frac{\Omega - \frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}}{\Delta t} = \frac{8\pi}{15\mathcal{I}_{\text{ext}}}r^4\rho\Omega U_2, \tag{52}$$

that immediately leads to:

$$\texttt{b1} = \frac{8\pi}{15\mathcal{I}_{\text{ext}}}r^4\rho\Omega U_2\Delta t - \left(\Omega - \frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}\right). \tag{53}$$

$\mathcal{I}_{\text{ext}}$ and $\mathcal{L}_{\text{ext}}$ are the values `xbext` and `xcext` computed in `confi`.

In case there is no external convective zone (*i.e.* the first layer of the interior is radiative), the boundary condition is simply:

$$\texttt{b1} = U_2. \tag{54}$$

`henadv` will then makes `b1` converge towards 0.

2.2.23 `ziadv`

`ziadv` is working exactly as `badv` in this version, except the "ext" labels have to be replaced by "int". In case there is a convective zone at the inner boundary, we have:

$$\texttt{az1} = \frac{8\pi}{15\mathcal{I}_{\text{int}}}r^4\rho\Omega U_2\Delta t - \left(\Omega - \frac{\mathcal{L}_{\text{int}}}{\mathcal{I}_{\text{int}}}\right). \tag{55}$$

$\mathcal{I}_{\text{int}}$ and $\mathcal{L}_{\text{int}}$ are the values `xbint` and `xcint` computed in `confi`.

In case the the centre is radiative, we have:

$$\texttt{az1} = U_2. \tag{56}$$

`henadv` will then makes `az1` converge towards 0.

2.2.24 `giadv`

This routine computes all the matrix elements and their derivatives, that are then used in `henadv` to solve the advection equation. An important point to be noted here is that, to account for the fact that the design of the Geneva code imposes that advection and diffusion are applied separately each two time steps, the radial component of the meridional circulation is here multiplied by a factor of 2 (or 1 + xcn at the first time step of a series). Georges, would you like to add some details here?

2.2.25 `coradv`

In this routine, the difference between the advected rotation profile `omegi` and the initial one (at the start of the computation of the angular momentum of the current structure iteration) `vvomeg` is computed, and stored in the variable `deladv`. We have thus, in most of the star:

$$\texttt{deladv} = \texttt{omegi} - \texttt{vvomeg}. \tag{57}$$

In case there is a convective zone at the inner (outer) boundary of the domain in which advection is properly computed, the angular velocity inside this region is computed as:

$$\texttt{deladv}_{\text{inside adjacent convective zones}} = \texttt{omegi}_{\text{inner/outer boundary}} - \frac{\mathcal{L}_{\text{in/out}}}{\mathcal{I}_{\text{in/out}}}, \tag{58}$$

where $\mathcal{I}_{\text{in/out}}$ and $\mathcal{L}_{\text{in/out}}$ comes from the values stored in `confi`.

2.2.26 omconv

The routine omconv aims to homogenise the angular velocity in the convective zones of the current iteration in henyey, given the new radial structure r. It proceeds as vomcon, by integrating in each convective zone the momentum of inertia $\mathcal{I}_{\rm CZ}$ and the angular momentum $\mathcal{L}_{\rm CZ}$. The new angular velocity of the convective zone is then defined as $\Omega_{\rm CZ} = \frac{\mathcal{L}_{\rm CZ}}{\mathcal{I}_{\rm CZ}}$. omegi is updated with these new angular velocities.

Note a small inconsistency here, omegi have been computed on the structure of the previous iteration rb, and re-ajusted here on the radial structure of the current one r.

2.2.27 omescale

The first action of this routine is to recompute the Ω-profile omegi. In henyey, it was systematically computed with the radial structure of the previous iteration rb. However, the final converged radial structure is r, and is slightly different. Local conservation of angular momentum is therefore used to modify omegi between these two structures:

$$\texttt{omegi}_i = \frac{\texttt{xMoCinScale}_i}{\mathcal{I}_i}. \tag{59}$$

This Ω-profile is then homogenised in the convective zones of the current model, and the total angular momentum of the model $\mathcal{L}_{\rm end}$, including an approximative enveloppe (see eq. 26), is computed.

The next step is the computation of the corrections that have to be applied on the Ω-profile to conserve angular momentum. This is done in the following way. First, the amount of angular momentum lost during the whole process so far is computed:

$$\Delta\mathcal{L}_{\rm computation} = \texttt{xLtotbeg} - \mathcal{L}_{\rm end}. \tag{60}$$

In a perfect world, this loss would be exactly equal to dlelex, the total amount of angular momentum lost through stellar winds. This is obviously not the case, so we define what remains to be lost:

$$\Delta\mathcal{L}_{\rm still\ to\ lose} = \texttt{dlelex} - \Delta\mathcal{L}_{\rm computation}. \tag{61}$$

The number of layers on which the correction will be applied is defined as:

$$N_{\rm corr,\ eff} = \begin{cases} \texttt{NPcoucheEff} & \text{if the 1st layer is radiative} \\ \min(\texttt{NPcoucheEff}, N_{\rm CZ\ depth}) & \text{if the 1st layer is convective and } N_{\rm CZ\ depth} \geq 25 \\ \texttt{NPcoucheEff} & \text{if the 1st layer is convective and } N_{\rm CZ\ depth} < 25 \end{cases}, \tag{62}$$

where $N_{\rm CZ\ depth}$ is the depth of the external convective zone (this means that the correction is applied *only* to the convective zone if it is deeper than 25 layers).

In the zone where the correction has to be applied, we assume that the corrected angular velocity $\Omega_{\rm corr}$ can be written as a function of the original Ω as (see Georgy 2010; Ekström et al. 2012):

$$\Omega_{\rm corr} = \Omega(1+q). \tag{63}$$

Then we simply impose the conservation of the angular momentum to get:

$$\begin{aligned} \mathcal{L}(\Omega) + \Delta\mathcal{L}_{\rm still\ to\ lose} &= \mathcal{L}(\Omega_{\rm corr}) \\ \mathcal{I}_{\rm env}\Omega_1 + \sum_{i=1}^{N_{\rm corr,\ eff}} \mathcal{I}_i\Omega_i + \Delta\mathcal{L}_{\rm still\ to\ lose} &= \mathcal{I}_{\rm env}\Omega_1 + \sum_{i=1}^{N_{\rm corr,\ eff}} \mathcal{I}_i\Omega_i + \mathcal{I}_{\rm env}q\Omega_1 + \sum_{i=1}^{N_{\rm corr,\ eff}} \mathcal{I}_i q\Omega_i, \end{aligned} \tag{64}$$

and thus, the correction factor is:

$$q = \frac{\Delta\mathcal{L}_{\rm still\ to\ lose}}{\mathcal{L}_{\rm env} + \sum_{i=1}^{N_{\rm corr,\ eff}} \mathcal{L}_i}, \tag{65}$$

where all the $\mathcal{L}$ are estimated with the non-corrected angular velocity Ω except $\mathcal{L}(\Omega_{\text{corr}})$. Then, the corrections are defined as:

$$\texttt{CorrOmega}_i = \begin{cases} q \cdot \texttt{omegi}_i & \text{if } 1 \leq i \leq N_{\text{corr, eff}} \\ 0 & \text{if } N_{\text{corr, eff}} < i \leq \texttt{NPcoucheEff} \end{cases}. \tag{66}$$

In case `rapcrilim` is set to 0, the correction `CorrOmega` are here set to 0 as well. In this case, the code behaves as it was before the modifications described here and in Georgy (2010).

Finally, we do not want this correction to break the constancy of Ω in the convective zones. To ensure this is not the case, the corrected profile Ω_{corr} is homogenised in the convective zones, leading to a profile $\Omega_{\text{corr, hom}}$. Then, the correction is redefined as:

$$\texttt{CorrOmega} = \Omega_{\text{corr, hom}} - \texttt{omegi}, \tag{67}$$

and the angular velocity outside the corrected zone as:

$$\texttt{omegi}_{\texttt{NPcoucheEff}<i\leq m} = \Omega_{\text{corr, hom}, i}. \tag{68}$$

This ensures that the Ω-profile outside the corrected zone is the right one, and that applying the correction in the corrected zone will lead to the correct value as well.

2.2.28 `om2old`

This routine computes the angular momentum for each shell inside the star, with the radial structure `r` (converged radius of the current time step), and `vomegi` (converged rotation profile of the current time step). The result is stored in `vxmoci`.

2.2.29 `omenex`

`omenex` computes the momentum of inertia of each layer $\mathcal{I}_{\texttt{rprov}}$ of the model, based on the estimated radial structure `rprov`. Then, a new rotation profile is computed by applying the local conservation of angular momentum, using `vxmoci` computed in `om2old`:

$$\texttt{omegi} = \frac{\texttt{vxmoci}}{\mathcal{I}_{\texttt{rprov}}}. \tag{69}$$

2.3 Questions about the design chosen a long, long time ago

- Why is the starting point of the computation of diffusion/advection the converged profile of the previous time step, and not the one of the current iteration, where the effects of local conservation of angular momentum (and convective zones) were already applied. Said in other words, why do we start `henadv/diffom` with `vvomeg(:)` instead of `omegp(:)`?

  The current way of doing is: we start from `vvomeg(:)`, we compute the effects of advection/diffusion and obtain a new Ω-profile `omegi(:)`. Then, the difference `deladv(:) = omegi(:) - vvomeg(:)` is computed, and the (assumed) final rotation profile is set to be `omegp(:) + deladv(:)`. This is of course a source of non-conservation of the angular momentum, even assuming a perfect conservation during the computation of the advection/diffusion itself.

- Why the radius structure used in the whole computation of advection/diffusion is taken from the *previous* iteration ? In other words, why do we use `rb(:)` instead of `r(:)`?

3 Treatment of rotation: new method

3.1 General philosophy

The idea of this new method is to make the treatment of rotation in the Geneva code more consistent with some basic assumptions imposed by the design of the code. The inclusion of the envelope in the computations in a more coherent way is also a major motivation for the changes made, as well as reducing the impact of the corrections that were added in Georgy (2010) to authorise a more accurate follow-up of the angular momentum. The angular momentum fluxes (due to stellar winds or magnetic braking) are also included as boundary conditions instead of being absorbed in the correction `CorrOmega` in the previous implementation. The momentum of inertia and the angular momentum of the envelope are also now correctly computed from the envelope data, and are no more only approximately estimated.

The equation to solve remains the same as previously:

$$\rho\partial_t \left(r^2\Omega\right)\big|_{M_r} = \frac{1}{5r^2}\partial_r \left(\rho r^4 \Omega U_2\right) + \frac{1}{r^2}\partial_r \left(\rho D_{\mathrm{v}} r^4 \partial_r \Omega\right), \tag{70}$$

with variables having the same meaning as in eq. (1).

By design, the Geneva code includes on top of the interior a so-called "envelope" that is assumed to rotate as a solid body at the same angular velocity as the first interior layer. This makes the boundary condition (3) inconsistent. Indeed, the envelope behaves, form the point of view of the transort of angular momentum, as a tiny convective zone always present on top of the interior, even if the first layer is radiative. To account for this fact, the two first boundary conditions (2, 3) become:

$$\partial_r \Omega = 0. \tag{71}$$

This is less physical than the old surface condition, but is more correct with the assumptions made for the rotation of the envelope.

The next boundary condition is also modified, to accept a possible flux of angular momentum:

$$\partial_t \left[\Omega \int_{r_t}^{R} r^4 \rho \mathrm{d}r\right] = -\frac{1}{5} r^4 \rho \Omega U_2 + \frac{3\dot{\mathcal{L}}_{\mathrm{excess}}}{8\pi}, \tag{72}$$

where $\dot{\mathcal{L}}_{\mathrm{excess}}$ is the total flux of angular momentum minus the isotropic contribution. Indeed, it is assumed in the Geneva code that the isotropic loss of angular momentum is carried away by the mass loss itself (as if we would remove a thin layer from the surface of a rotating solid sphere: the removal of angular momentum is exactly compensated by the removal of mass).

Finally, the same condition at the centre is not changed:

$$\partial_t \left[\Omega \int_{r_{\mathrm{inf,\,CZ}}}^{r_b} r^4 \rho \mathrm{d}r\right] = -\frac{1}{5} r^4 \rho \Omega U_2. \tag{73}$$

The way the envelope is now included in the computation of the transport of angular momentum is discussed below.

3.1.1 Resolution of the diffusion equation

Two important changes have been made compared to the version described in Section 2.1.1. The envelope is now included as an additional layer accounted for in the computations, and it is possible to account for a flux of angular momentum at the surface.

Inclusion of the envelope: The envelope is considered as a virtual additional layer in the computation. To do so, the needed vectors (*e.g* Ω, r, ...) are extended below the first layer by adding a 0th component containing the data of the envelope. Knowing the mass M_{env} of the envelope and its momentum of inertia $\mathcal{I}_{\text{env}}$, an average envelope radius r_{env} is defined as the radius that would have a spherical shell with the same momentum of inertia:

$$r_{\text{env}} = \sqrt{\frac{3\mathcal{I}_{\text{env}}}{2M_{\text{env}}}}. \tag{74}$$

The average density of the envelope is defined as:

$$\rho_{\text{env}} = \frac{3M_{\text{env}}}{4\pi\left(R_\star^3 - r_1^3\right)}, \tag{75}$$

where $R_\star$ is the stellar radius and r_1 the radius of the first layer of the interior. To ensure that the envelope and the first layer of the interior will still rotating at the same angular velocity, a very strong diffusion coefficient is imposed in between these layers $D_{\text{envelope/interior}} = 10^{20}$.

Addition of a flux term at the surface: The flux of angular momentum is added at the surface the following way. Equation (10) is rewritten **for the envelope layer**:

$$\frac{3}{2}\mathcal{I}\partial_t\Omega = \left[4\pi\rho D_{\text{v}} r^4 \partial_r\Omega\right]_{r-\Delta r/2}^{r+\Delta r/2} + \frac{3}{2}\dot{\mathcal{L}}_{\text{excess}}. \tag{76}$$

Following Section 2.1.1, the linear equation for the surface (12) is rewritten:

$$\text{For k = 0 (surface)} : \Omega_0^{\text{n}} + \frac{\dot{\mathcal{L}}_{\text{excess}}}{\mathcal{I}_0}\Delta t = \Omega_1^{\text{n+1}}\frac{[1,0]}{\frac{3}{2}\mathcal{I}_0} + \Omega_0^{\text{n+1}}\left(1 - \frac{[1,0]}{\frac{3}{2}\mathcal{I}_0}\right). \tag{77}$$

This system is then solved the same way as before.

3.1.2 Resolution of the advection equation

The changes here concerne only the boundary conditions, the general system of equations being solved remaining the same as in Section 2.1.2. As mentioned above, the boundary conditions for Θ are now systematically $\Theta = 0$. In order to account for the envelope, the external boundary condition (72) is now implemented. The integration on the left-hand-side now includes the envelope, and the right-hand-side includes the flux term. Note that contrarily to what was done in the old version, in case the first layer of the interior is radiative, the envelope is still considered as a small convective zone, and thus the boundary condition (72) always applies.

3.1.3 Conservation errors as a new flux

An important improvement we propose in this new version is the possibility to account for the angular momentum conservation errors as a new external flux. As will be shown later (Section 5), the new advective or diffusive treatment of the transport equation (accounting for the envelope) leads in most of the case to a very good conservation of the angular momentum. Most of the errors come from the assumption that the isotropic mass loss carries away its own angular momentum (*i.e.* that the structure does not readjust to the mass loss).

With the new possibility to directly add a flux to the boundary conditions of both advection and diffusion equation, we propose to reinject the difference between the conservative case and the computed one as an additional flux added to `xldoex` ($\dot{\mathcal{L}}_{\text{excess}}$) in equations (72, 77). During the first iteration, this flux is set to 0, and we use for the next iterations the difference in the previous iteration. The detailed is explained below in the concerned subroutines.

The major advantage of this method is that it considerably reduces the errors that have to be corrected with `CorrOmega` as in the old method, avoiding this artificial correction in an arbitrary number of shells below the surface. The corrections `CorrOmega` are still active in the new version to ensure an exact conservation, but they are now much smaller (see Section 5).

3.2 Detailed description of the general algorithm and involved routines

This section is almost a copy-paste of the similar one of the first part. In the title, aside the name of the routine is indicated its status in this new version (untouched, modified, new). All the modifications with respect to the previous version are indicated in green boldface. A general picture of the new version is shown on Fig. 2.

3.2.1 `main` (modified)

The first thing made by the code in link with rotation is the reading of the binary file from the previous time-steps series. Eight important variables are read at this point: `r`, `vr`, `omegi`, `vomegi`, `CorrOmega`, `xLtotbeg`, `vsuminenv`, `dlelexprev`. At this moment, `r` is the converged radius of the previous time step, and `vr` the converged radius of the antepenultimate one. `omegi` is the estimated rotation profile of the current time step, `vomegi` the converged one of the previous time step. **`vsuminenv` is the momentum of inertia of the envelope of the converged model of the previous time step.**

After that, the number of layers `NPcoucheEff` on which the correction is applied below the surface is set, either to 200 for the main sequence, or to 50 for the advanced stages.

Once this is done, the code enters the main loop, which loops over the wanted number of time steps computed during the sequence (usually, 10, except during the very advanced stages when it is automatically put to 20 to reduce the number of structure files produced).

Here starts the main resolution loop in `main`. If a model has convergence problems, it will restart from here (see below). In case the value of `fitm` has changed, the routine `fitmshift` is called.

Then, `VcritCalc` is called, providing values such as the ratio $\omega = \frac{\Omega}{\Omega_{\text{crit, 1}}}$, $\omega_2 = \frac{\Omega}{\Omega_{\text{crit, 2}}}$, the equatorial velocity, the oblateness, *etc*. At this point, these are estimated values, based on the estimated temperature, luminosity, and surface angular velocity interpolated from the previous time step (non converged values).

The next step from there is the computation of the increase of the mass-loss rate due to rotation, stored in the variable `alpro6`. A detailed explanation of this computation can be found in Maeder & Meynet (2000), and some tests and correction of an error in Georgy (2010).

In the new version, the call to `momevo` is brought outside the condition on the parity of the time step, and returns `BTotalStartModel` the total angular momentum of the converged model of the previous time step, including the envelope (with `vsuminenv`). At this point, there is a branching depending on whether advection is applied or not during the current time step. In case `iadvec` is 1 and the time step number is even (meaning that diffusion is applied during the time step), or in case `iadvec` is 0 and `idiff` is 1 (meaning the diffusion only is applied, typically, when magnetic fields are accounted for, or during the advanced stages), **`xLtotbeg` takes the value of `BTotalStartModel`, reproducing the behaviour of the previous version**.

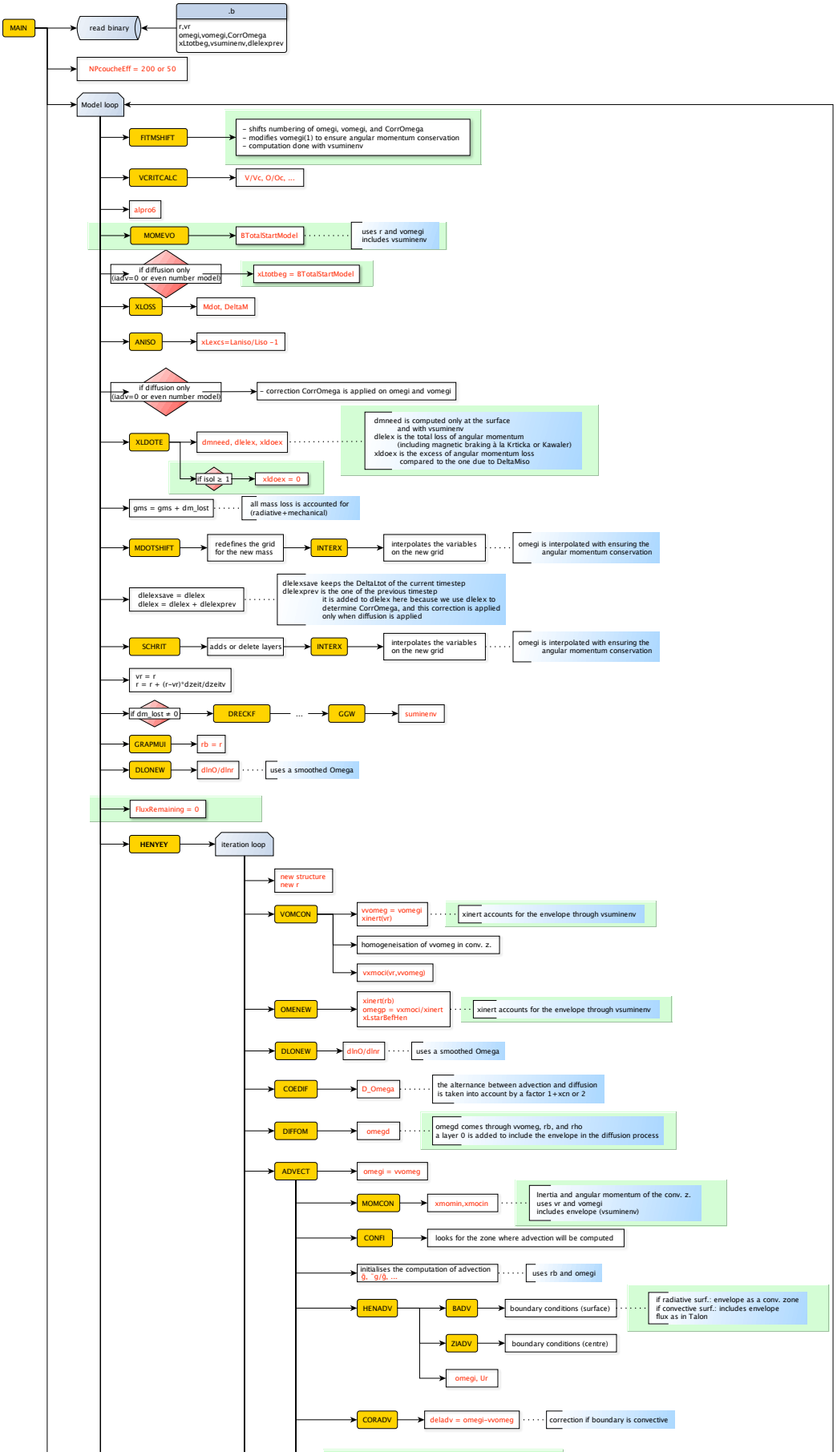
Then, `xloss` is called, computing the mass-loss rate `xmdot` as a function of stellar parameters (T_{eff}, L, surface chemical composition, *etc.*). Note that this mass-loss rate is computed on the basis of estimated parameters for the time step (non converged). This mass-loss rate is then converted into a total mass lost during the time step, `dm_lost`, and is then removed from the total mass `gms`. Once this is done, `aniso` is called, returning the values `bdotis` and `xlexcs`.

The next step is again only in case `iadvec` is 1 and the time step number is even or in case `iadvec` is 0 and `idiff` is 1: in this situation, the rotation profiles `vomegi` (converged profile at the previous time step) and `omegi` (estimated rotation profile at the current time step) are corrected by `CorrOmega`[7]:

$$\begin{aligned} \texttt{vomegi} &= \texttt{vomegi} + \texttt{CorrOmega}, \\ \texttt{omegi} &= \texttt{omegi} + \texttt{CorrOmega}. \end{aligned} \tag{78}$$

A test ensures that the angular velocity remains positive there. In case it would become negative, it is reset to 10^{-20}. In principle, it never occurs, since this is already tested elsewhere (see below). At this

[7]Note that formally, the correction is computed from `vomegi` (see below), and it is thus only an approximation to apply it directly to `omegi`. It provides however a better estimate for the angular momentum content of the star.


MAIN
read binary
.b
r,vr
omegi,vomegi,CorrOmega
xLtotbeg,vsuminenv,dlelexprev
NPcoucheEff = 200 or 50
Model loop
FITMSHIFT
- shifts numbering of omegi, vomegi, and CorrOmega
- modifies vomegi(1) to ensure angular momentum conservation
- computation done with vsuminenv
VCRITCALC
V/Vc, O/Oc, ...
alpro6
MOMEVO
BTotalStartModel
uses r and vomegi
includes vsuminenv
if diffusion only
(iadv=0 or even number model)
xLtotbeg = BTotalStartModel
XLOSS
Mdot, DeltaM
ANISO
xLexcs=Laniso/Liso -1
if diffusion only
(iadv=0 or even number model)
- correction CorrOmega is applied on omegi and vomegi
XLDOTE
dmneed, dlelex, xldoex
dmneed is computed only at the surface
and with vsuminenv
dlelex is the total loss of angular momentum
(including magnetic braking à la Krticka or Kawaler)
xldoex is the excess of angular momentum loss
compared to the one due to DeltaMiso
if isol ≥ 1
xldoex = 0
gms = gms + dm_lost
all mass loss is accounted for
(radiative+mechanical)
MDOTSHIFT
redefines the grid
for the new mass
INTERX
interpolates the variables
on the new grid
omegi is interpolated with ensuring the
angular momentum conservation
dlelexsave = dlelex
dlelex = dlelex + dlelexprev
dlelexsave keeps the DeltaLtot of the current timestep
dlelexprev is the one of the previous timestep
it is added to dlelex here because we use dlelex to
determine CorrOmega, and this correction is applied
only when diffusion is applied
SCHRIT
adds or delete layers
INTERX
interpolates the variables
on the new grid
omegi is interpolated with ensuring the
angular momentum conservation
vr = r
r = r + (r-vr)*dzeit/dzeitv
if dm_lost ≠ 0
DRECKF
...
GGW
suminenv
GRAPMUI
rb = r
DLONEW
dlnO/dlnr
uses a smoothed Omega
FluxRemaining = 0
HENYEY
iteration loop
new structure
new r
VOMCON
vvomeg = vomegi
xinert(vr)
xinert accounts for the envelope through vsuminenv
homogeneisation of vvomeg in conv. z.
vxmoci(vr,vvomeg)
OMENEW
xinert(rb)
omegp = vxmoci/xinert
xLstarBefHen
xinert accounts for the envelope through vsuminenv
DLONEW
dlnO/dlnr
uses a smoothed Omega
COEDIF
D_Omega
the alternance between advection and diffusion
is taken into account by a factor 1+xcn or 2
DIFFOM
omegd
omegd comes through vvomeg, rb, and rho
a layer 0 is added to include the envelope in the diffusion process
ADVECT
omegi = vvomeg
MOMCON
xmomin,xmocin
Inertia and angular momentum of the conv. z.
uses vr and vomegi
includes envelope (vsuminenv)
CONFI
looks for the zone where advection will be computed
initialises the computation of advection
g, ´g/g, ...
uses rb and omegi
HENADV
BADV
boundary conditions (surface)
if radiative surf.: envelope as a conv. zone
if convective surf.: includes envelope
flux as in Talon
ZIADV
boundary conditions (centre)
omegi, Ur
CORADV
deladv = omegi-vvomeg
correction if boundary is convective

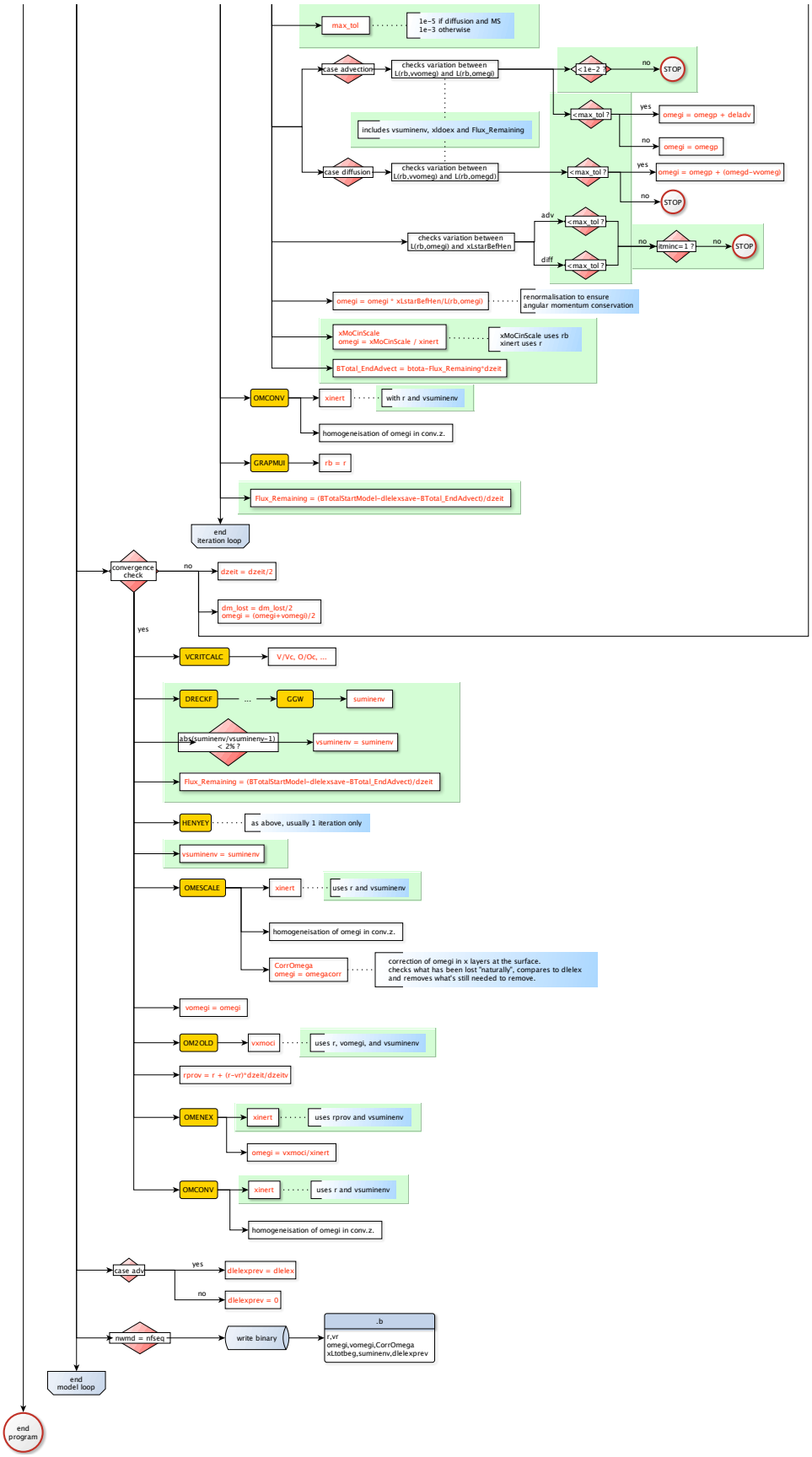


Figure 2: Flowchart of the treatment of rotation in the new version proposed here. The changes with respect to the 2013 version (Fig. 1) are indicated by the green boxes.

point, we also check if the model has finished the main sequence. If this is the case, `CorrOmega` is completely reset to 0, since the code will switch to a fully diffusive behaviour, where the correction is applied at each time step. The number of layers on which the correction is computed `NPcoucheEff` is switched to 50.

Then, there is a very important call to the routine **`xldote`**, which returns the total angular momentum loss `dlelex`, the non-isotropic contribution to this loss `xldoex`, as well as an estimate of the mechanical equatorial mass loss when the star rotates critically: `dmneed`. Once this value is known, the total mass of the star is corrected: `gms` = `gms` + `dmneed`. In case the mechanical or radiative mass loss is not zero, `MdotShift` is called, to redefine the mesh points according to the new total mass.

To keep a correct track of the angular momentum loss, `dlelex` is then updated: `dlelex` = `dlelex` + `dlelexprev`. `dlelexprev` is null if the correction of the rotation profile was applied during the previous time step. Otherwise, it contains the angular momentum loss during the previous time step. At this point, `dlelex` contains either the angular momentum loss of the current time step, or of the two previous ones. The angular momentum loss of the current time step *only* is saved in `dlelexsave`.

The mesh is adapted according to some criteria (that can be set in the parameter card), through the routine `schrit`.

Then, an approximative model is built by interpolating the radius from the two previous time steps:

$$\begin{aligned} \mathtt{r} &= \mathtt{r} + \frac{\Delta t}{\Delta t_{\text{prev}}}\left(\mathtt{r} - \mathtt{vr}\right), \\ \mathtt{vr} &= \mathtt{r}. \end{aligned} \tag{79}$$

The radius `r` is at this point saved in the variable `rb`, in the routine `grapmui`. **`Flux_remaining` is initialised to 0 at this point.**

The next step is the resolution of the structure equations themselves, which is done in the **`henyey`** routine. At the end of the routine, the convergence of the structure is checked. In case the model is not converged, all the converged data of the previous time step are reinitialised, and the estimated one of the current time step are recomputed as the average between the converged and the "old-estimated" ones, for example:

$$\mathtt{omegi} = \frac{\mathtt{vomegi} + \mathtt{omegi}}{2}, \tag{80}$$

and the time step, mass loss, *etc.* are divided by 2. The code restarts the computation of the model with this reduced time step just before the call to `fitmshift` here above.

In case the model is correctly converged, the various critical velocities, the oblateness, and the polar and equatorial radii are recomputed on the basis of the converged model by a new call to `VcritCalc`. Then, **the momentum of inertia of the envelope is computed in the routines integrating the envelope, and saved in `suminenv`. If the variation of the momentum of inertia of the envelope compared to the previous time step is less than 2%, then `vsuminenv` is updated with `suminenv`. However, in case the variation is too big, it is not updated, since it causes frequent crashes in `henyey` since it produces a big variation of the angular momentum of the envelope. `Flux_remaining` is updated with the data from the last iteration in `henyey`:**

$$\mathtt{Flux_remaining} = \frac{\overbrace{\mathtt{BTotal_StartModel} - \mathtt{dlelexsave}}^{\mathcal{L}\text{ that should be reached at the end of }\Delta t} - \overbrace{\mathtt{BTotal_EndAdvect}}^{\mathcal{L}\text{ at the end of advection/diffusion}}}{\Delta t}. \tag{81}$$

`henyey` is called again a last time to compute the final fully converged structure, and print it in the v-file if requested. At this point, `vsuminenv` is updated with the momentum of inertia of the envelope of the current time step `suminenv` in any cases, and contains now the converged value.

The next major routine for the conservation of the angular momentum is the call to **`omescale`**, in charge of computing the corrections `CorrOmega` to be applied to the first `NPcoucheEff` layers below the surface. Once this is done, the converged Ω-profile is saved in `vomegi`:

$$\mathtt{vomegi} = \mathtt{omegi}. \tag{82}$$

The momentum of inertia of each shell is computed in **om2old**, and stored in the variable vxmoci.

What remains in main is the computation of an estimate of the rotation profile at the next time step. First, the radial structure of the next time step is estimated:

$$\texttt{rprov} = \texttt{r} + \frac{\Delta t}{\Delta t_{\text{prev}}}(\texttt{r} - \texttt{vr}). \tag{83}$$

Then, a new rotation profile omegi is computed assuming the local conservation of angular momentum between the radial structure r and rprov in the routine **omenex**. This new Ω-profile is then homogenised in the convective zones with another call to **omconv**.

The last step about the treatment of the transport of angular momentum in main is the saving of dlelexprev. If the current model number is odd, meaning that the advection was applied, then dlelexprev is set to dlelex (the angular momentum lost during the current time step is saved). In the other case, dlelexprev is reinitialised to 0.

Finally, if the current model number is the last of a series, all the relevant data about the rotation and angular momentum transport are saved in the binary file: r, vr, omegi, vomegi, CorrOmega, xLtotbeg, vsuminenv, abd dlelexprev. In case a new model has to be computed, the whole process is repeated, starting again just before the call to fitmshift here above. Otherwise, the execution stops.

3.2.2 fitmshift (modified)

This routine aims to manage the mass meshing when fitm is changed in the course of a computation. **It has been improved with respect to the previous version by including the envelope in a more coherent way, and by accounting for the fact that changing fitm redistributes angular momentum between the interior and the envelope. The treatment has been improved as well in case fitm is increasing.**

In case fitm is decreasing, the following steps are performed:

1. The number of layers that have to be removed, n_0, is computed.

2. **The fraction of the layer which the new fitm falls in that remains in the envelope is computed:**

$$f_{\textbf{envelope}} = \frac{1 - \exp(\mathbf{q}_{n_0}) - \mathbf{fitm}}{\exp(\mathbf{q}_{n_0+1}) - \exp(\mathbf{q}_{n_0})}, \tag{84}$$

   where n_0 is the last old mesh coordinate above the new fitm.

3. The angular momentum contained in the removed layers is computed, accounting for the envelope:

$$\mathcal{L}_{\text{old}} = \mathcal{I}_{\text{env, old}}(\Omega_{1,\,\text{old}} + \Delta\Omega_{1,\,\text{old}}) \sum_{i=0}^{n_0+1} \mathcal{I}_{i,\,\text{old}}(\Omega_{i,\,\text{old}} + \Delta\Omega_{i,\,\text{old}}), \tag{85}$$

   where is indices "old" refers to the old numbering of the layers (see below). Note here the following points:

   - In the Geneva code, the momentum of inertia of each layer of the interior, except the first one and last one, is given by:

$$\mathcal{I}_i = \frac{2}{3}\frac{M_{i-1} - M_{i+1}}{2} r_i^2. \tag{86}$$

     For the surface, it is:

$$\mathcal{I}_1 = \frac{2}{3}\frac{M_1 - M_2}{2} r_1^2, \tag{87}$$

     and for the centre:

$$\mathcal{I}_m = \frac{2}{5}\frac{M_{m-1}}{2} r_m^2. \tag{88}$$

For the envelope, we take in this version the value stored in `vsuminenv`:

$$\mathcal{I}_{\text{env}} = \texttt{vsuminenv}. \tag{89}$$

and we assume that it is rotating at the same velocity as the first layer of the interior.

Due to the way the mass of a layer is computed (involving the $n-1$the layer), we need to consider all the layers up to the n_0+1th one in the sum of Eq. 85.

- At this point, `omegi` contains the non corrected value of the angular velocity. We thus need to account for the correction `CorrOmega` in order to get the correct angular momentum.

4. **The momentum of inertia of the envelope is corrected, absorbing the momentum of inertia of each removed layers and the corresponding fraction if the one partially removed:**

$$\mathcal{I}_{\text{env, new}} = \mathcal{I}_{\text{env, old}} + \sum_{i=1}^{n_0} \mathcal{I}_i + f_{\text{envelope}} \mathcal{I}_{n_0+1}. \tag{90}$$

5. The numbering of the layers is shifted by n_0: $n_0 \to 1$, $n_0+1 \to 2$, *etc.* The mass coordinate of the first layer is set to `fitm`.

6. We adjust the angular velocity of the first layer and the envelope by imposing:

$$\mathcal{L}_{\text{old}} = \mathcal{I}_{\text{env, new}} \Omega_{1,\text{ new}} + \sum_{i=1}^{2} \mathcal{I}_{i,\text{ new}} \Omega_{i,\text{ new}}. \tag{91}$$

This leads to the following relation for $\Omega_{1,\text{ new}}$:

$$\Omega_{1,\text{ new}} = \frac{\mathcal{L}_{\text{old}} - \mathcal{I}_2 \Omega_{2,\text{ new}}}{\mathcal{I}_{1,\text{ new}} + \mathcal{I}_{\text{env, new}}}. \tag{92}$$

This modification is absorbed in the correction of the angular velocity:

$$\Delta\Omega_{1,\text{new}} = \Omega_{1,\text{ new}} - \Omega_{n_0,\text{ old}}. \tag{93}$$

In case `fitm` is increasing, we assume angular momentum conservation between the previous `fitm` and the new one. First, the mass coordinate of the first layer $M_{1,\text{ old}}$ before `fitm` has changed is saved. We then write, assuming increasing `fitm` does not change the velocity of the envelope or of the first layer:

$$\begin{aligned}
(\Omega_{1,\text{ old}} + \Delta\Omega_{1,\text{ old}})\,(\mathcal{I}_{\text{env, old}} + \mathcal{I}_{1,\text{ old}} + \mathcal{I}_{2,\text{ old}}) &= (\Omega_{1,\text{ old}} + \Delta\Omega_{1,\text{ old}})\,(\mathcal{I}_{\text{env, new}} + \mathcal{I}_{1,\text{ new}} + \mathcal{I}_{2,\text{ new}}) \\
\mathcal{I}_{\text{env, old}} + \frac{2}{3}\frac{M_{1,\text{ old}} - M_2}{2} r_1^2 + \frac{2}{3}\frac{M_{1,\text{ old}} - M_3}{2} r_2^2 &= \mathcal{I}_{\text{env, new}} + \frac{2}{3}\frac{M_{1,\text{ new}} - M_2}{2} r_1^2 + \frac{2}{3}\frac{M_{1,\text{ new}} - M_3}{2} r_2^2 \\
\mathcal{I}_{\text{env, new}} &= \mathcal{I}_{\text{env, old}} + \frac{1}{3}\left(r_1^2 + r_2^2\right)\left(M_{1,\text{ old}} - M_{1,\text{ new}}\right),
\end{aligned} \tag{94}$$

where we have used relations above for the $\mathcal{I}_i$. By design, only the M_1's are changing while increasing `fitm`, and the radii $r_{1,2}$ remain constant as well.

3.2.3 `VcritCalc` (untouched)

This routine computes several quantities linked with the surface rotation, such as the oblateness, the ratios $\Omega/\Omega_{\text{crit, 1}}$, $\Omega/\Omega_{\text{crit, 2}}$, $v/v_{\text{crit, 1}}$, and $v/v_{\text{crit, 2}}$, the polar and equatorial radii, the equatorial linear velocity. The computation is done starting from the normalised value of the stellar surface, used to interpolate the oblateness in the file `ftfp.dat`. The velocity used here is the current value stored in `omegi`.

The inverse of the oblateness `f` is computed as well, and used again later. The details of the computations can be found in Ekström et al. (2008) and Maeder (2009).

3.2.4 `momevo` (modified)

`momevo` computes the total angular momentum of the star. It takes four input parameters, a radii profile, an angular velocity profile, the correction applied to the angular velocity, and a logical. It returns the total angular momentum of the star, computed as in `fitmshift` (but obviously through all the layers). If the logical is `True`, then the envelope is *not* included in the total. Otherwise, it is included, **with its momentum of inertia stored in `vsuminenv`**.

3.2.5 `xloss` (untouched)

This routine is in charge of the computation of the mass-loss rate at different phase of the evolution of the star. A complete description of the different recipes used here can be found in Georgy (2010).

3.2.6 `aniso` (untouched)

In this routine, the shape of the surface of the star is computed as a function of the current `f` parameter of the star. This allows one to compute for each colatitude on the star the local temperature, gravity, and fraction of the mass flux emerging at this point. It is then rescaled to the total value of the mass loss `dm_lost`. Then, the angular momentum flux is computed at each colatitude and integrated over the surface. A complete description of this procedure can be found in Maeder (2002), Georgy (2010), and Georgy et al. (2011).

Finally, the routine returns the excess/lack of angular momentum loss with respect to the isotropic case `xlexcs`, and the value of the angular momentum lost if we assume it is isotropic `bdotis`.

3.2.7 `xldote` (modified)

The routine `xldote` computes several quantities linked with the angular momentum loss, and provides an estimate of the equatorial "mechanical" mass loss needed in order to keep the star at the critical velocity when it reaches it.

Estimate of the mechanical mass loss. The account for the mechanical mass loss is controlled by the parameter `rapcrilim`. If it is set to 0, then the computation of this mechanical mass loss is jumped. Otherwise, `rapcrilim` defines the maximal ratio $\texttt{rapcrilim} = \omega_{\rm lim} = \frac{\Omega_{\rm lim}}{\Omega_{\rm crit}}$ authorised for the star. It is by default set to 0.99, meaning that the code will compute the mechanical mass loss needed to keep the star at this fraction of the critical velocity. First of all, the maximal authorised angular velocity is computed (in the equations shown in this section, when a quantity is first define, we also write `in teletype font` the name of the variable in the code. We then use only the mathematical symbol to save space[8].):

$$\Omega_{\rm lim} = \texttt{omlim} = \omega_{\rm lim}\frac{\Omega_1}{\omega}. \tag{95}$$

The total angular momentum loss during the current time step is also computed (accounting for the effects of the anisotropic stellar wind if needed):

$$\Delta\mathcal{L} = \texttt{dlelex} = -\texttt{bdotis}(1+\texttt{xlexcs}). \tag{96}$$

Contrarily to the previous version, we do *not* assume here that the angular momentum is lost from the `NPcoucheEff` first layers of the star, but only to the envelope and first layer. This has to be tested a bit more. Formally it should work, but maybe it would require too small time steps. **The procedure to estimate the mechanical mass-loss rate is exactly the same as in the 2013 version of the code, except this number of layers. We assume the surface velocity will change as:**

$$\Omega_1^{\rm new} = \Omega_1\,(1+q)\,, \tag{97}$$

[8]Moreover, the following notations are used: Ω_i is the angular velocity of the ith layer, ω is the ratio $\frac{\Omega_1}{\Omega_{\rm crit}}$ where $\Omega_{\rm crit}$ is the critical velocity.

we can show by imposing the total angular momentum conservations that q is (Georgy 2010; Georgy et al. 2013):

$$q = \frac{\mathcal{L}_{\text{e, ini}} \frac{\Delta M_{\text{rad}}}{m_{\text{e, ini}}} - \Delta\mathcal{L}}{\mathcal{L}_{\text{tot, ini}} - \mathcal{L}_{\text{e, ini}} \frac{\Delta M_{\text{rad}}}{m_{\text{e, ini}}}}, \tag{98}$$

where

$\mathcal{L}_{\text{tot, ini}} = \mathcal{L}_{\text{e, ini}} + \mathcal{L}_{1,\text{ ini}}$ **(where the subscript "e" means "envelope"),**

$\mathcal{L}_{\text{tot, fin}} = \mathcal{L}_{\text{e, ini}} (1+q) \left(1 - \frac{\Delta M_{\text{rad}}}{m_{\text{e, ini}}}\right) + (1+q)\,\mathcal{L}_1,$

ΔM_{rad} is the total mass loss during the time step,

$m_{\text{e, ini}}$ the initial mass of the envelope.

The estimated surface angular velocity becomes thus:

$$\Omega_1^{\text{new}} = \texttt{newomega} = \Omega_1(1+q). \tag{99}$$

Then, this value is compared to Ω_{lim}. In case $\Omega_1^{\text{new}} > \Omega_{\text{lim}}$, the mechanical mass loss `dmneed` is estimated as follows (Georgy 2010; Georgy et al. 2013):

$$\Delta M_{\text{mech}} = \texttt{dmneed} = \frac{\mathcal{L}_{\text{tot, ini}} \left(1 - \frac{\Omega_{\text{lim}}}{\Omega_{1,\text{ ini}}}\right) + \mathcal{L}_{\text{e, ini}} \frac{\Delta M_{\text{rad}}}{m_{\text{e, ini}}} \frac{\Omega_{\text{lim}}}{\Omega_{1,\text{ ini}}} - \Delta\mathcal{L}}{\Omega_{1,\text{ ini}} r_{\text{eq}}^2 - \frac{\mathcal{L}_{\text{e, ini}}}{m_{\text{e, ini}}} \frac{\Omega_{\text{max}}}{\Omega_{1,\text{ ini}}}}, \tag{100}$$

with r_{eq} the equatorial radius of the star, which is computed in `aniso`. We assume here that this mass is lost at the equator in a circumstellar disc. In case ω is significantly higher than ω_{lim} at the beginning of the time step, this mass loss is multiplied by a factor higher than 1 (depending how far from ω_{lim} we are). The corresponding angular momentum loss `dLmeca` is thus:

$$\Delta\mathcal{L}_{\text{mech}} = \texttt{dLmeca} = \Delta M_{\text{mech}} \Omega_{\text{surf}} r_{\text{eq}}^2. \tag{101}$$

Total angular momentum loss. In a next step, the routine `dLmagcalc` is called, returning the angular momentum loss due to magnetic braking `dLmag` (by construction of the formula used, `dLmag` contains the contribution of both the usual isotropic loss due to radiative winds and the contribution from the the magnetic braking). **A magnetic braking following the Kawaler (1988) law is possibly computed in `dLmagLM`, leading to an additional angular momentum loss stored in `dL_Kawaler`.** The total angular momentum loss during the time step is recomputed as:

$$\texttt{dlelex} = \underbrace{\texttt{dLmag}}_{\text{iso. winds + mag. brak.}} - \underbrace{\texttt{bdotis} \cdot \texttt{xlexcs}}_{\text{anisotropic winds contribution}} + \underbrace{\texttt{dLmeca}}_{\text{mechanical mass loss}} + \underbrace{\texttt{dL_Kawaler}}_{\text{Kawaler braking}}, \tag{102}$$

and the angular momentum in excess compared to the isotropic case **used as an outflux in the computation of advection/diffusion is:**

$$\dot{\mathcal{L}}_{\text{excess}} = \texttt{xldoex} = -\frac{\texttt{dlelex} + \texttt{bdotis}}{\Delta t}, \tag{103}$$

(be careful, the signs are not the same for `dlelex` and `bdotis`).

3.2.8 `dLmagcalc` (untouched)

This routine takes as input the isotropic angular momentum loss during the time step, and returns its value when a given magnetic breaking is applied (or return the same value in case magnetic braking is not accounted for). The formalism is the same as in Meynet et al. (2011) and Ud-Doula et al. (2009):

$$\dot{\mathcal{L}}_{\text{mag}} = \frac{2}{3} \dot{M} \Omega_1 R_\star^2 \left[0.29 + (\eta_\star + 0.25)^{\frac{1}{4}}\right]^2, \tag{104}$$

where $R_\star$ is the stellar radius, and $\eta_\star = \frac{B_{\rm eq}^2 R_\star^2}{\dot{M} v_\infty}$. $B_{\rm eq}$ is the magnetic field at the equator at the stellar surface, and v_∞ is the terminal wind velocity. For estimating this velocity, we use the parametrisation from Kudritzki & Puls (2000):

$$v_\infty = C\left(T_{\rm eff}\right) v_{\rm esc}, \tag{105}$$

with

$$C\left(T_{\rm eff}\right) = \begin{cases} 2.65 & T_{\rm eff} > 21000\,[\mathrm{K}] \\ 1.4 & 10000 < T_{\rm eff} \leq 21000\,[\mathrm{K}] \\ 1 & T_{\rm eff} \leq 10000\,[\mathrm{K}] \end{cases}. \tag{106}$$

3.2.9 `dLmagLM` (new)

This function computes the loss of angular momentum during the current time step, due to magnetic braking according to a Kawaler (1988) law (see also Eggenberger et al. 2008). The loss is computed as:

$$\frac{\mathrm{d}\mathcal{L}_{\text{mag. brak.}}}{\mathrm{d}t} = \begin{cases} -K\Omega_{\rm surf}^3 \left(\frac{R}{R_\odot}\right)^{\frac{1}{2}} \left(\frac{M}{M_\odot}\right)^{-\frac{1}{2}} & \Omega_{\rm surf} < \Omega_{\rm sat} \\ -K\Omega_{\rm surf}\Omega_{\rm sat}^2 \left(\frac{R}{R_\odot}\right)^{\frac{1}{2}} \left(\frac{M}{M_\odot}\right)^{-\frac{1}{2}} & \Omega_{\rm surf} > \Omega_{\rm sat} \end{cases}, \tag{107}$$

where K and $\Omega_{\rm sat}$ are free parameters, that appear in the parameter card as `K_Kawaler` and `Omega_saturation` and can be changed there. In case `K_Kawaler` is zero, the function returns 0 as well. Otherwise, it returns $\frac{\mathrm{d}\mathcal{L}_{\text{mag. brak.}}}{\mathrm{d}t}\Delta t$.

3.2.10 `MdotShift` (untouched)

This routine is used to redefine the mass mesh `q` inside the star once the mass has been adjusted due to mass loss. It also interpolates the values at the centre and the surface. In more details:

- The position of the new `fitm` is found in the old grid. The first layer is redefine at this position, and all the quantities are interpolated at this point. Concerning `vomegi` and `omegi`, see the note below.
- In case the new `fitm` falls *not* between the first and second layer of the old meshing, the required layers are removed.
- The layer number $m-1$ is redefined and the values interpolated on its new position.
- The mass coordinate `q` (which depends on the total mass) is redefined everywhere, in order to keep the *real mass* position constant.

Important note: In case of very strong mass loss, more than one layer might have to be removed from the interior. In case a very strong gradient of Ω is present in this region, the interpolation process makes the surface rotate suddenly very fast (usually above the critical limits), such that the code crashes. This is due to the fact that the surface rotates implicitly at the same angular velocity as the first layer of the interior. To improve this situation, the following is now done in this routine. By design, `MdotShift` removes a fraction `fitm` of the total mass loss in the interior (in fact, it repositions $q(1)$ inside the star, and removes the possible layers of the previous models that are above this value), and a fraction 1-`fitm` in the envelope. So, we assume that:

1. the velocity of the envelope should not change: it remains at the same velocity $\Omega_{1,\,\rm prev}$ as before, since the mass removed from the envelope carries out its own angular momentum, and thus does not affect the surface angular velocity.
2. the angular velocity of the first layer inside the star takes an interpolated value $\Omega_{1,\,\rm interp}$. To stay in the design of the code, we would require that the angular velocity of the envelope $\Omega_{1,\,\rm prev}$, and the velocity of the first layer $\Omega_{1,\,\rm interp}$ are the same. However, it has no reason to be true.

So a new surface (and first layer) velocity is computed, imposing angular momentum conservation and equalising the angular velocity of both zones. Technically, it is done as follows. The momentum of inertia and the angular momentum of the envelope and first layer are[9]:

$$\begin{aligned}
\mathcal{I}_{\rm e} &= \frac{2}{3}M_{\rm e}r_1^2 = \frac{2}{3}(1-\texttt{fitm})M_{\rm tot}r_1^2 = \frac{2}{3}\exp(q_1)M_{\rm tot}r_1^2,\\
\mathcal{L}_{\rm e} &= \mathcal{I}_{\rm e}\Omega_{1,\,\rm prev},\\
\mathcal{I}_1 &= \frac{2}{3}M_1r_1^2 = \frac{2}{3}(\exp(q_2)-\exp(q_1))M_{\rm tot}r_1^2,\\
\mathcal{L}_1 &= \mathcal{L}_1\Omega_{1,\,\rm interp}.
\end{aligned} \tag{108}$$

The new surface angular velocity is then reset to:

$$\Omega_{1,\,\rm new} = \frac{\mathcal{L}_{\rm e}+\mathcal{L}_1}{\mathcal{I}_{\rm e}+\mathcal{I}_1}. \tag{109}$$

3.2.11 `schrit` (untouched)

This routine parses the mesh, and ensures that the number of radial zones is enough according to some criteria (some of them can be set in the parameter card). In case there is too much zones at some places, they are removed. In case the discretisation is too coarse, layers are added and the other ones are renumbered. This is done with ensuring the conservation of the total angular momentum.

3.2.12 `grapmui` (untouched)

`grapmui` computes the mean molecular weight gradient ∇_μ. In case `idialu` is set to 1, this gradient is smoothed, producing cleaner profiles to be used for the computation of the advection.

Importantly for the computation of the angular momentum, the radius at the previous iteration `rb` is saved in this routine as well.

3.2.13 `dlonew` (untouched)

This routine computes the value of $\frac{\mathrm{d}\ln(\Omega)}{\mathrm{d}\ln(r)}$. The values of the current iteration `omegi` and `r` are used. If `idialo` is set to 1, it computes a smoothed profile of Ω and $\frac{\mathrm{d}\ln(\Omega)}{\mathrm{d}\ln(r)}$.

3.2.14 `henyey` (modified)

`henyey` is the main routine used to solve the structure equations. It uses a relaxation method, proposed by Henyey et al. (1964). The purpose of this document is not to describe how this is done, and the details can be found in any textbook about stellar evolution (*e.g.* Kippenhahn & Weigert 1990; Maeder 2009). By design, the Geneva code does not solve simultaneously the structure equations, the change in the chemical composition (due to nuclear burning and transport processes) and the transport of angular momentum. It proceeds step by step, looping as follows:

1. one iteration for solving the structure is done;
2. the transport and burning of chemical species is computed, as well as the transport of angular momentum;
3. the convergence of the structure is tested. In case it has not converged, the loop is reiterated.

The following gives a detailed explanation about the transport of angular momentum resolution. Immediately after the first part of the routine, we thus have a new structure, particularly new radii `r` (and also, *e.g.* new convective zones, *etc.*). At this stage, the routine **`vomcon`** is called, homogenising

[9]These momentums are computed in the same way as in `fitmshift`.

the angular velocity in the convective zones. Moreover, it returns the angular momentum of each layers vxmoci, **the envelope being now included in the first layer**.

Once this is done, the code computes the new rotation profile on the structure of the previous iteration (`rb`) by applying the local conservation of the angular momentum. This is done by the subroutine **`omenew`**, which moreover returns the total angular momentum ***including the envelope*** `xLstarbefHen`. The derivative $\mathrm{d}\ln(\Omega)/\mathrm{d}\ln r$ is recomputed (with `omegi` and `r`) in `dlonew`.

The various diffusion coefficients are then computed in `coedif`. With them, the diffusion of the angular momentum is computed in **`diffom`**, which returns a new rotation profile `omegd`. In a next step, the routine **`advect`** is called. Finally, the Ω-profile is homogenised in the convective zones in the routine **`omconv`**. Then, `grapmui` is called again, and stores the radial structure `r` in `rb` for the next structure iteration.

At this point, the residual flux that will be reinjected in the next iteration of `henyey` is computed. It si defined as the angular momentum that has to be reached at the end of the time step, minus the angular momentum effectively obtained at the end of the advection/diffusion process, divided by the time step:

$$\texttt{Flux_remaining} = \frac{\overbrace{\texttt{BTotal_StartModel} - \texttt{dlelexsave}}^{\mathcal{L}\text{ that should be reached at the end of }\Delta t} - \overbrace{\texttt{BTotal_EndAdvect}}^{\mathcal{L}\text{ at the end of advection/diffusion}}}{\Delta t}. \quad (110)$$

This angular momentum flux is reinjected in the boundary conditions during the computation of the advection in `henadv` or of the diffusion in `diffom`.

The whole process starts again from the beginning of `henyey`, until the structure is converged or the maximum number of iterations is reached.

3.2.15 `vomcon` (modified)

This routine makes three different operations on the `vomegi` profile (let us recall here that `vomegi` is here the converged Ω-profile at the end of the previous time step):

- `vomegi` is saved in the variable `vvomeg`.
- `vvomeg` is homogenised in the *current* convective zones. To proceed, the code computes in each convective zone the total momentum of inertia $\mathcal{I}_{\mathrm{CZ}}$ and angular momentum $\mathcal{L}_{\mathrm{CZ}}$, and redefines Ω in these regions as $\Omega_{\mathrm{CZ}} = \frac{\mathcal{L}_{\mathrm{CZ}}}{\mathcal{I}_{\mathrm{CZ}}}$. **The momentum of inertia of the envelope is accounted for in case there is an external convective zone (with `vsuminenv`). This reduces the error on the conservation of the angular momentum, by automatically making the envelope rotates at the same angular velocity as the convective zone).**
- The angular momentum of each layer, computed with the radii `vr` (*i.e.* the converged radius of the previous time step) and the angular velocities `vomegi` (*i.e.* the converged Ω-profile of the previous time step) is stored in `vxmoci`. **The value of `vxmoci` for the first layer includes the envelope (with `vsuminenv`).**

3.2.16 `omenew` (modified)

This routine computes the new Ω profile by applying the local conservation of the angular momentum. The starting point is `vxmoci` computed in `vomcon`. This angular momentum is divided by the momentum of inertia computed with the radii of the *previous* iteration in `henyey` (or the estimated radii `rb` computed in `main` during the first iteration). The new Ω-profile is thus computed as:

$$\texttt{omegp} = \frac{\texttt{vxmoci}\,(\texttt{vr}, \texttt{vomegi})}{\mathcal{I}\,(\texttt{rb})}. \quad (111)$$

The momentum of inertia of the envelope `vsuminenv` is included in the first layer of $\mathcal{I}\,(\texttt{rb})$, to improve conservation of the angular momentum and for consistency with `vomcon`.

To summarise, we thus have:

$$\left.\begin{array}{l}\overbrace{\begin{array}{ll}\text{converged radius} & \texttt{vr}\\ \text{converged } \Omega & \texttt{vomegi}\end{array}\Big\}\texttt{vxmoci}}^{\text{previous time step}}\\[2ex] \underbrace{\text{previous iteration radius}\quad \texttt{rb}\}\,\mathcal{I}(\texttt{rb})}_{\text{current time step}}\end{array}\right\}\Rightarrow \texttt{omegp}\ \text{(local angular mom. cons.)}$$

At the end of the routine, the total angular momentum of **of the model, including the envelope,** is stored in the variable `xLstarbefHen`. It will be used later on to check the conservation of angular momentum during the advection/diffusion computation.

3.2.17 `coedif` (untouched)

This routine computes the various diffusion coefficient linked with the transport of chemical species and angular momentum. We do not aim in this document at providing a detailed description of what is done here (see *e.g.* Maeder 1997; Talon & Zahn 1997; Zahn 1992; Maeder 2003; Mathis et al. 2004, or the code documentation available through svn.). The choice of the diffusion coefficient is done through the parameter `icoeff` of the parameter card, and the inclusion of the so-called "Tayler-Spruit" dynamo (Spruit 2002) in its revised version (Maeder & Meynet 2003, 2004, 2005) is controlled by the parameter `imagn` of the parameter card. The diffusion coefficients are stored in the variables `D_chim` for the chemical species, and `D_Omega` for the angular momentum.

An important point to be noted here is that, because the design of the Geneva code imposes advection and diffusion being applied separately each two time steps, the diffusion coefficient is here multiplied by a factor of 2 (or 1 + xcn at the first time step of a series).

3.2.18 `diffom` (modified)

The routine `diffom` computes the diffusion of the angular momentum, with the diffusion coefficient computed in `coedif`. **As described in Section 3.1.1, the envelope is accounted for as a 0th layer, with a mean density, mass, and radius determined in order to reproduce the momentum of inertia of the envelope `vsuminenv`. The out flux of excess of angular momentum `xldoex` is considered in the boundary condition. In case we are *not* at the first iteration of the first call to `henyey`, `Flux_remaining` is added as an out/in flux the same way `xldoex` (see Section 3.1.3). This considerably improves the angular momentum conservation, making the total angular momentum after the computation of the diffusion already contains the gain/loss of the current time step.** The numerical method is an implicit finite differences scheme (see Sect. 2.1.1 and 3.1.1). The time step is divided in a number of sub-step, according to the Courant condition. The matrix inversion is done by the mean of an LU decomposition, which is very efficient thanks to the tridiagonal property of the system (see Press et al. 2007). Schematically, we have here:

$$\left.\begin{array}{l}\overbrace{\begin{array}{ll}\textbf{radius} & \textbf{rb}\\ \textbf{diffusion coefficients} & \\ \textbf{"non-conserved" flux} & \textbf{Flux_remaining}\end{array}}^{\textbf{previous iteration}}\\[2ex] \underbrace{\textbf{converged } \Omega\quad \textbf{vvomeg}}_{\textbf{previous time step}}\\[2ex] \underbrace{\Delta\mathcal{L}_{\text{excess}}\quad \textbf{xldoex}}_{\textbf{current time step}}\end{array}\right\}\Rightarrow \textbf{omegd}$$

3.2.19 `advect` (modified)

This routine contains quite a lot of different cases. Let us examine them one by one.

In case `iadvec` is 1: the advection is computed. This needs several pre-computations. First, `omegi` is re-initialised to `vvomeg`. The momentum of inertia `xmomin` and the angular momentum `xmocin` of each convective zones are computed in **`momcon`**. Then, the configuration of the convective zones inside the star is checked, and the momentum of inertia and angular momentum of the relevant ones are provided in `xbint`, `xcint`, `xbext`, and `xcext` by a call to the routine `confi`.

In case the advection needs to be computed (*i.e.* `inoadv` has not been set to 1 in `confi`), the code then calls a set of routines initialising various quantities useful for the advection, such as $\frac{\tilde{g}}{g}$, *etc.* They are computed using the structure of the previous iteration `rb`, and the Ω-profile stored in `omegi`.

Once this is done, the advection part of the angular momentum transport equation (16) is solved using a relaxation scheme (see Section 3.1.2) by calling the routine **`henadv`**. At the end of the process, we get `omegi` the rotation profile solution of the equation. This solution is then applied in the whole star (including the zones where advection was not computed) in the routine `coradv`, which returns the *difference* between the final solution of the advection `omegi` and the initial profile `vvomeg`, stored in the variable `deladv`.

Then, the code behaves again differently depending on the parity of the time step number:

- *If the time step is odd (advection applied):* The conservation of the angular momentum during the computation of advection is checked: **the total angular momentum (with envelope) computed with the structure `rb` and rotation profile `vvomeg`, plus the in/out fluxes `xldoex` and `Flux_remaining`, and computed with `rb` and `omegi` are compared. In case it varies more than 10^{-2} and we are during the last `henyey` call, the execution is stopped in any case**. In case it varies more than 10^{-3} (relatively), or the maximum number of iterations was reached when computing the advection in `henadv`, **and we are during the last `henyey` call**, then the computation of advection is rejected[10]. Otherwise, it is accepted. At this stage, the *actual* rotation profile is corrected as follows:

$$\texttt{omegi} = \begin{cases} \texttt{omegp} + \texttt{deladv} & \text{if accepted} \\ \texttt{omegp} & \text{if rejected} \end{cases}.$$

  (Note that the meaning of `omegi` changes here: before, it was the actual solution of the advection equation starting from the the initial profile `vvomeg`, and from now on, it is the profile where the local conservation of angular momentum was applied, and corrected for the effects of advection through `deladv`.)

  Then, the total angular momentum **(including the envelope)** is again recomputed with the new `omegi` and `rb`, and compared with the angular momentum content after the local conservation of angular momentum **plus the in/out fluxes `xldoex` and `Flux_remaining`**, computed with `rb` and `omegp`. In case the relative variation is bigger than 10^{-3}, the execution of the code stops.

[10] During the very first models at the beginning of the Main Sequence, where the Ω-profile has to adjust quite a lot, the computation of advection is accepted, even if the variation is bigger than 10^{-3}. This is possible only while the central H fraction is in the range $[X_{\rm ini} - 5 \cdot 10^{-3}, X_{\rm ini}]$.

Schematically, these steps are summarised as:

$$\left.\begin{array}{c}\mathbf{vvomeg}\\ \mathbf{rb}\\ \mathcal{L}_{\text{bef. henadv}}\end{array}\right\}\xrightarrow{\text{henadv}}\left\{\begin{array}{c}\Omega_{\text{advected}}\\ \mathbf{rb}\\ \mathcal{L}_{\text{after henadv}}\end{array}\right\}\rightarrow\begin{array}{c}\mathbf{deladv}=\\ \Omega_{\text{advected}}-\mathbf{vvomeg}\end{array}\left\{\begin{array}{c}\underbrace{\mathbf{omegi}=\mathbf{omegp}+\mathbf{deladv}}_{\text{if accepted}}\\ \mathbf{rb}\\ \mathcal{L}_{\text{final}}\\ \\ \underbrace{\mathbf{omegi}=\mathbf{omegp}}_{\text{if rejected}}\\ \mathbf{rb}\\ \mathcal{L}_{\text{final}}\end{array}\right\}$$

$$\mathcal{L}_{\text{final}}\ \textbf{compared with}\ \mathbf{xLstarbefHen}+(\mathbf{xldoex}+\mathbf{Flux_remaining})\,\Delta t\left\{\begin{array}{c}\textbf{accepted}\\ \\ \textbf{rejected}\rightarrow\textbf{stop}\end{array}\right.$$

- *If the time step is even (diffusion applied):* The conservation of the angular momentum during the computation of diffusion is checked: the total angular momentum **(with envelope)** computed with the structure `rb` and rotation profile `vvomeg`, **plus the in/out fluxes `xldoex` and `Flux_remaining`**, and computed with `rb` and `omegd` are compared. In case it varies more than 10^{-5} (during the main sequence) or 10^{-3} (during the advanced phases, relatively) then the computation of diffusion is rejected and the execution stopped. Otherwise, it is accepted. At this stage, the *actual* rotation profile is corrected as follows:

$$\texttt{omegi}=\texttt{omegp}+\texttt{omegd}-\texttt{vvomeg}.$$

Then, the total angular momentum **(with envelope)** is again recomputed with the new `omegi` and `rb`, and compared with the angular momentum content after the local conservation of angular momentum, **plus the in/out fluxes `xldoex` and `Flux_remaining`**, computed with `rb` and `omegp`. In case the relative variation is bigger than 10^{-5}, the execution of the code stops.

Schematically, these steps are summarised as:

$$\left.\begin{array}{c}\texttt{vvomeg}\\ \texttt{rb}\\ \mathcal{L}_{\text{bef. diffom}}\end{array}\right\}\xrightarrow{\texttt{diffom}}\left\{\begin{array}{c}\texttt{omegd}\\ \texttt{rb}\\ \mathcal{L}_{\text{after diffom}}\end{array}\right\}\left\{\begin{array}{c}\underbrace{\texttt{omegi}=\texttt{omegp}+\texttt{omegd}-\texttt{vvomeg}}_{\text{if accepted}}\\ \texttt{rb}\\ \mathcal{L}_{\text{final}}\\ \\ \text{stop if rejected}\end{array}\right\}$$

$$\mathcal{L}_{\text{final}}\ \text{compared with}\ \texttt{xLstarbefHen}+(\texttt{xldoex}+\texttt{Flux_remaining})\,\Delta t\left\{\begin{array}{c}\text{accepted}\\ \\ \text{rejected}\rightarrow\text{stop}\end{array}\right.$$

In both cases, the Ω-profile are rescaled in order to ensure the conservation of angular momentum:

$$\texttt{omegi}=\texttt{omegi}\,\frac{\texttt{xLstarbefHen}+(\texttt{xldoex}+\texttt{Flux_remaining})\,\Delta t}{\mathcal{L}_{\text{final}}}. \tag{112}$$

and the angular momentum of each layers of the star, computed with `rb` and the rescaled `omegi` are stored in the variable `xMoCinScale`. This rescaling is mandatory due to the design of the code. Indeed, even with an advection/diffusion scheme that would perfectly conserve angular momentum,

since the input Ω-profile is `vvomeg` and *not* `omegp`, there is no reason why the step from `omegp` to `omegi` described above would conserve it.

In a next step, the Ω-profile, so far computed with the radius of the previous iteration `rb`, is transformed by applying the local angular momentum conservation on the last radial structure `r`:

$$\mathtt{omegi}_i = \frac{\mathcal{L}_i(\mathtt{rb})_i}{\mathcal{I}_i(\mathtt{r})_i}. \tag{113}$$

The total angular momentum reached by the model at the end of the transport process (*without* accounting for the residual `Flux_remaining`) is saved:

$$\mathtt{BTotal_EndAdvect} = \sum_{i=1}^{m} \mathcal{L}_i + \mathcal{L}_{\mathrm{env}} - \mathtt{Flux_remaining} \cdot \Delta t. \tag{114}$$

In case `iadvec` is 0: Advection is not computed, and the code directly jumps to the part described in the second item above (case where diffusion is applied).

3.2.20 `momcon` (modified)

For each convective zone inside the star, the total momentum of inertia is computed and stored in `xmomin`, and the total angular momentum is computed and stored in `xmocin`. They are both computed with the converged values of the radii and the angular velocities at the previous time step `vr` and **`vvomeg`**. These quantities are computed as usual elsewhere in the code. **The momentum of inertia of the envelope `vsuminenv` and its angular momentum are accounted for in the external convective zone in case the first layer is convective.**

3.2.21 `confi` (untouched)

`confi` stores in `xbint`, `xcint`, `xbext`, and `xcext` the values of the momentum of inertia and of the angular momentum of the relevant convective zones, according to a convective configuration computed in `rechzco` (not described here). The following cases are considered:

0. The star is entirely convective. In this case, advection is not considered, and `inoadv` is set to 1.
1. The star is entirely radiative. The advection is computed from the centre to the surface.
2. The star has a radiative centre and surface, but has some intermediate convective zones. In this case, advection is computed between the surface and the first convective zone encountered going toward the centre.
3. The star is convective at the centre and surface, and has other convective zones in between. In this case, advection is applied in the first radiative zone below the surface.
4. The star is radiative at the centre and convective at the surface. In this case, advection is applied in the first radiative zone below the surface (possibly going up to the centre).
5. The star is convective at the centre and radiative at the surface. Advection is considered from the surface to the first convective zone encountered going toward the centre.

3.2.22 `henadv` (modified)

This routine aims to solve the advective part of the angular momentum transport equation (16), by mean of a relaxation scheme similar to the one used in `henyey` to solve the structure equation. It starts with the rotation profile `vvomeg`, and iteratively solves the equation up to the final solution, which is then stored in `omegi`.

The boundary conditions (72, 73) are defined in **badv** for the surface, and `ziadv` for the centre. In case The boundary condition for $\partial_r\Omega$ (71) is expressed as a function of Θ:

$$\Theta_1 = 0. \tag{115}$$

The matrix elements for the interior are computed in `giadv`.

Schematically, we have:

$$\left.\overbrace{\begin{array}{ll}\text{radius} & \texttt{rb}\\ \Omega\text{-profile} & \texttt{omegi}\end{array}}^{\text{previous iteration}}\right\} \Rightarrow \text{converged } \texttt{omegi}$$

3.2.23 badv (modified)

This routine computes the matrix element called "`b1`" and its derivatives, that encodes the boundary condition (72). If there is an external convective boundary, the boundary condition is rewritten as follows. First, we use relation (9) to get:

$$\frac{3}{8\pi}\partial_t\left(\Omega\mathcal{I}_{\text{ext}}\right) = -\frac{1}{5}r^4\rho\Omega U_2 + \frac{3(\dot{\mathcal{L}}_{\text{excess}} + \dot{\mathcal{L}}_{\text{remaining}})}{8\pi}, \tag{116}$$

where $\dot{\mathcal{L}}_{\text{remaining}}$ is stored in `Flux_remaining` and is the same flux difference between the expected final total angular momentum and the one we got (see Sections 3.1.3 and 3.2.18), and is set to 0 at the first iteration of a time step. We then assume that only Ω is varying during the time step (the background structure is fixed), and that the angular velocity of the previous time step is equal to $\frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}$ (in order to account for the possible change of the convective zones). We have thus:

$$\frac{\Omega - \frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}}{\Delta t} = \frac{8\pi}{15\mathcal{I}_{\text{ext}}}r^4\rho\Omega U_2 + \frac{\dot{\mathcal{L}}_{\text{excess}} + \dot{\mathcal{L}}_{\text{remaining}}}{\mathcal{I}_{\text{ext}}}, \tag{117}$$

that immediately leads to:

$$\texttt{b1} = \left(\frac{8\pi}{15\mathcal{I}_{\text{ext}}}r^4\rho\Omega U_2 + \frac{\dot{\mathcal{L}}_{\text{excess}} + \dot{\mathcal{L}}_{\text{remaining}}}{\mathcal{I}_{\text{ext}}}\right)\Delta t - \left(\Omega - \frac{\mathcal{L}_{\text{ext}}}{\mathcal{I}_{\text{ext}}}\right). \tag{118}$$

$\mathcal{I}_{\text{ext}}$ and $\mathcal{L}_{\text{ext}}$ are the values `xbext` and `xcext` computed in `confi` in case the first layer of the interior is convective. In case it is radiative, they correspond to `vsuminenv` and `vsuminenv` $\cdot$ `vvomeg`$_1$.

`henadv` will then makes `b1` converge towards 0.

3.2.24 ziadv (untouched)

`ziadv` is working exactly as `badv` in this version, except the "ext" labels have to be replaced by "int". In case there is a convective zone at the inner boundary, we have:

$$\texttt{az1} = \frac{8\pi}{15\mathcal{I}_{\text{int}}}r^4\rho\Omega U_2\Delta t - \left(\Omega - \frac{\mathcal{L}_{\text{int}}}{\mathcal{I}_{\text{int}}}\right). \tag{119}$$

$\mathcal{I}_{\text{int}}$ and $\mathcal{L}_{\text{int}}$ are the values `xbint` and `xcint` computed in `confi`.

In case the the centre is radiative, we have:

$$\texttt{az1} = U_2. \tag{120}$$

`henadv` will then makes `az1` converge towards 0.

3.2.25 `giadv` (untouched)

This routine computes all the matrix elements and their derivatives, that are then used in `henadv` to solve the advection equation. An important point to be noted here is that, to account for the fact that the design of the Geneva code imposes that advection and diffusion are applied separately each two time steps, the radial component of the meridional circulation is here multiplied by a factor of 2 (or 1 + xcn at the first time step of a series). Georges, would you like to add some details here?

3.2.26 `coradv` (untouched)

In this routine, the difference between the advected rotation profile `omegi` and the initial one (at the start of the computation of the angular momentum of the current structure iteration) `vvomeg` is computed, and stored in the variable `deladv`. We have thus, in most of the star:

$$\texttt{deladv} = \texttt{omegi} - \texttt{vvomeg}. \tag{121}$$

In case there is a convective zone at the inner (outer) boundary of the domain in which advection is properly computed, the angular velocity inside this region is computed as:

$$\texttt{deladv}_{\text{inside adjacent convective zones}} = \texttt{omegi}_{\text{inner/outer boundary}} - \frac{\mathcal{L}_{\text{in/out}}}{\mathcal{I}_{\text{in/out}}}, \tag{122}$$

where $\mathcal{I}_{\text{in/out}}$ and $\mathcal{L}_{\text{in/out}}$ comes from the values stored in `confi`.

3.2.27 `omconv` (modified)

The routine `omconv` aims to homogenise the angular velocity in the convective zones of the current iteration in `henyey`, given the new radial structure `r`. It proceeds as `vomcon`, by integrating in each convective zone the momentum of inertia $\mathcal{I}_{\text{CZ}}$ and the angular momentum $\mathcal{L}_{\text{CZ}}$. **The momentum of inertia of the envelope and its angular momentum is added to the one of the first layer of the interior.** The new angular velocity of the convective zone is then defined as $\Omega_{\text{CZ}} = \frac{\mathcal{L}_{\text{CZ}}}{\mathcal{I}_{\text{CZ}}}$. `omegi` is updated with these new angular velocities.

3.2.28 `omescale` (modified)

First, the total angular momentum of the model $\mathcal{L}_{\text{end}}$, including the envelope, is computed. The next step is the computation of the corrections that have to be applied on the Ω-profile to conserve angular momentum. This is done in the following way. First, the amount of angular momentum lost during the whole process so far is computed:

$$\Delta\mathcal{L}_{\text{computation}} = \texttt{xLtotbeg} - \mathcal{L}_{\text{end}}. \tag{123}$$

In a perfect world, this loss would be exactly equal to `dlelex`, the total amount of angular momentum lost through stellar winds. This is obviously not the case, so we define what remains to be lost:

$$\Delta\mathcal{L}_{\text{still to lose}} = \texttt{dlelex} - \Delta\mathcal{L}_{\text{computation}}. \tag{124}$$

The number of layers on which the correction will be applied is defined as:

$$N_{\text{corr, eff}} = \begin{cases} \texttt{NPcoucheEff} & \text{if the 1st layer is radiative} \\ \min(\texttt{NPcoucheEff}, N_{\text{CZ depth}}) & \text{if the 1st layer is convective and } N_{\text{CZ depth}} \geq 25 \\ \texttt{NPcoucheEff} & \text{if the 1st layer is convective and } N_{\text{CZ depth}} < 25 \end{cases}, \tag{125}$$

where $N_{\text{CZ depth}}$ is the depth of the external convective zone (this means that the correction is applied *only* to the convective zone if it is deeper than 25 layers).

In the zone where the correction has to be applied, we assume that the corrected angular velocity Ω_{corr} can be written as a function of the original Ω as (see Georgy 2010; Ekström et al. 2012):

$$\Omega_{\text{corr}} = \Omega(1 + q). \tag{126}$$

Then we simply impose the conservation of the angular momentum to get:

$$\mathcal{L}(\Omega) + \Delta\mathcal{L}_{\text{still to lose}} = \mathcal{L}(\Omega_{\text{corr}})$$

$$\mathcal{I}_{\text{env}}\Omega_1 + \sum_{i=1}^{N_{\text{corr, eff}}} \mathcal{I}_i\Omega_i + \Delta\mathcal{L}_{\text{still to lose}} = \mathcal{I}_{\text{env}}\Omega_1 + \sum_{i=1}^{N_{\text{corr, eff}}} \mathcal{I}_i\Omega_i + \mathcal{I}_{\text{env}}q\Omega_1 + \sum_{i=1}^{N_{\text{corr, eff}}} \mathcal{I}_i q\Omega_i, \tag{127}$$

and thus, the correction factor is:

$$q = \frac{\Delta\mathcal{L}_{\text{still to lose}}}{\mathcal{L}_{\text{env}} + \sum_{i=1}^{N_{\text{corr, eff}}} \mathcal{L}_i}, \tag{128}$$

where all the $\mathcal{L}$ are estimated with the non-corrected angular velocity Ω except $\mathcal{L}(\Omega_{\text{corr}})$. Then, the corrections are defined as:

$$\texttt{CorrOmega}_i = \begin{cases} q \cdot \texttt{omegi}_i & \text{if } 1 \leq i \leq N_{\text{corr, eff}} \\ 0 & \text{if } N_{\text{corr, eff}} < i \leq \texttt{NPcoucheEff} \end{cases}. \tag{129}$$

In case `rapcrilim` is set to 0, the correction `CorrOmega` are here set to 0 as well. In this case, the code behaves as it was before the modifications described here and in Georgy (2010).

Finally, we do not want this correction to break the constancy of Ω in the convective zones. To ensure this is not the case, the corrected profile Ω_{corr} is homogenised in the convective zones, leading to a profile $\Omega_{\text{corr, hom}}$. Then, the correction is redefined as:

$$\texttt{CorrOmega} = \Omega_{\text{corr, hom}} - \texttt{omegi}, \tag{130}$$

and the angular velocity outside the corrected zone as:

$$\texttt{omegi}_{\texttt{NPcoucheEff}<i\leq m} = \Omega_{\text{corr, hom}, i}. \tag{131}$$

This ensures that the Ω-profile outside the corrected zone is the right one, and that applying the correction in the corrected zone will lead to the correct value as well.

3.2.29 `om2old` (modified)

This routine computes the angular momentum for each shell inside the star, with the radial structure `r` (converged radius of the current time step), and `vomegi` (converged rotation profile of the current time step). **The envelope is included in the first shell.** The result is stored in `vxmoci`.

3.2.30 `omenex` (modified)

`omenex` computes the momentum of inertia of each layer $\mathcal{I}_{\texttt{rprov}}$ of the model, based on the estimated radial structure `rprov`. **Again, the envelope is included in the first layer.** Then, a new rotation profile is computed by applying the local conservation of angular momentum, using `vxmoci` computed in `om2old`:

$$\texttt{omegi} = \frac{\texttt{vxmoci}}{\mathcal{I}_{\texttt{rprov}}}. \tag{132}$$

4 Summary of current limitations (both implementations)

- The advection is computed only in the first radiative zone below the surface.
- After the main sequence, the computation of advection is probably not correct. A term has to be modified in the routine `giadv`, in the computation of the terms of the so-called "equation 3" (check for the comment `! attention epsi mu correct seulement pour H burn`).
- So far, the momentum of inertia and the angular momentum inside the star are computed as if the shells were spherical. This could be improved by adding a column in the ftfp file were the geometrical number appearing in the momentum of inertia (*e.g.* $\frac{2}{3}$ for a spherical shell) could be precomputed for a given ratio to the critical velocity.

- When computing with a large number of shells (with the typical number introduced to compute with rotation since the 2010 version of the code, typically), it happens that the diffusion coefficients are jiggling a lot. It seems that this is due to the increase in the spatial resolution rather than the introduction of `rapcrilim` and the corrections on the Ω-profile `CorrOmega`. In some cases, it has a huge impact on the transport of chemicals (primary N, for example). This has to be solved.

5 Comparisons between the two methods

6 Comparison with previous computations

6.1 Fast rotating intermediate mass star

Here we compare the evolution of a $7\,M_{\odot}$ star at solar metallicity, with an initial rotation rate $\omega = 0.7$. The computation is performed up to the end of the central He-burning. The comparison is made with a similar star computed in the framework of the "Be-grid" (Georgy et al. 2013). Figure 3 illustrates the excellent behaviour of the new version compared to previous computations. Except unavoidable very small differences, the general agreement is excellent (even the spike during He burning is almost exactly reproduced). This confirms that:

- the new implementation is robust compared to previous results and does not have obvious bugs,
- the results published so far with the angular velocity correction only (`Origin2013` version) to ensure angular momentum conservation, even if the correction was made slightly unphysically, are correct, in the sense that the mixing of chemical species and angular momentum transport are well reproduced with the new version, that have a more physical implementation of the conservation.

Finally, in this case of low mass-loss rate, the new method is as costly as the old one in terms of time steps during the evolution of the star.

6.2 Comparison between different implementations

In this section, we compare various versions of the implementations discussed here above. We distinguish the following five cases:

- A model computed with the `2013` version of the code, *i.e.* corresponding to the implementation discussed in Section 2. They are shown in black in the following plots.
- A model computed with the new implementation discussed in Section 3, but **without reintroducing the residual as an external flux** (*i.e.* the variable `Flux_remaining` is set to 0), and **the corrections of the surface angular velocity for the angular momentum conservation are not accounted for** (*i.e.* the variable `rapcrilim` in the parameter card is set to 0). We this expect a bad conservation of the angular momentum in this case, as both controls are deactivated. It corresponds to the situation before the modifications made in Georgy (2010), except that the envelope is better accounted for here. This model is in red in the plots.
- A model computed with the new implementation discussed in Section 3, **without reintroducing the residual as an external flux** (*i.e.* the variable `Flux_remaining` is set to 0), but **with the correction of the surface angular velocity accounted for**. It is the same implementation as the `2013` version, except that the envelope is included in the treatment of rotation, and the boundary conditions are changed. This model is in green in the following plots.
- A model computed with the new implementation discussed in Section 3, **including the reinjection of the residuals** as an external flux. This model has **no correction of the surface**

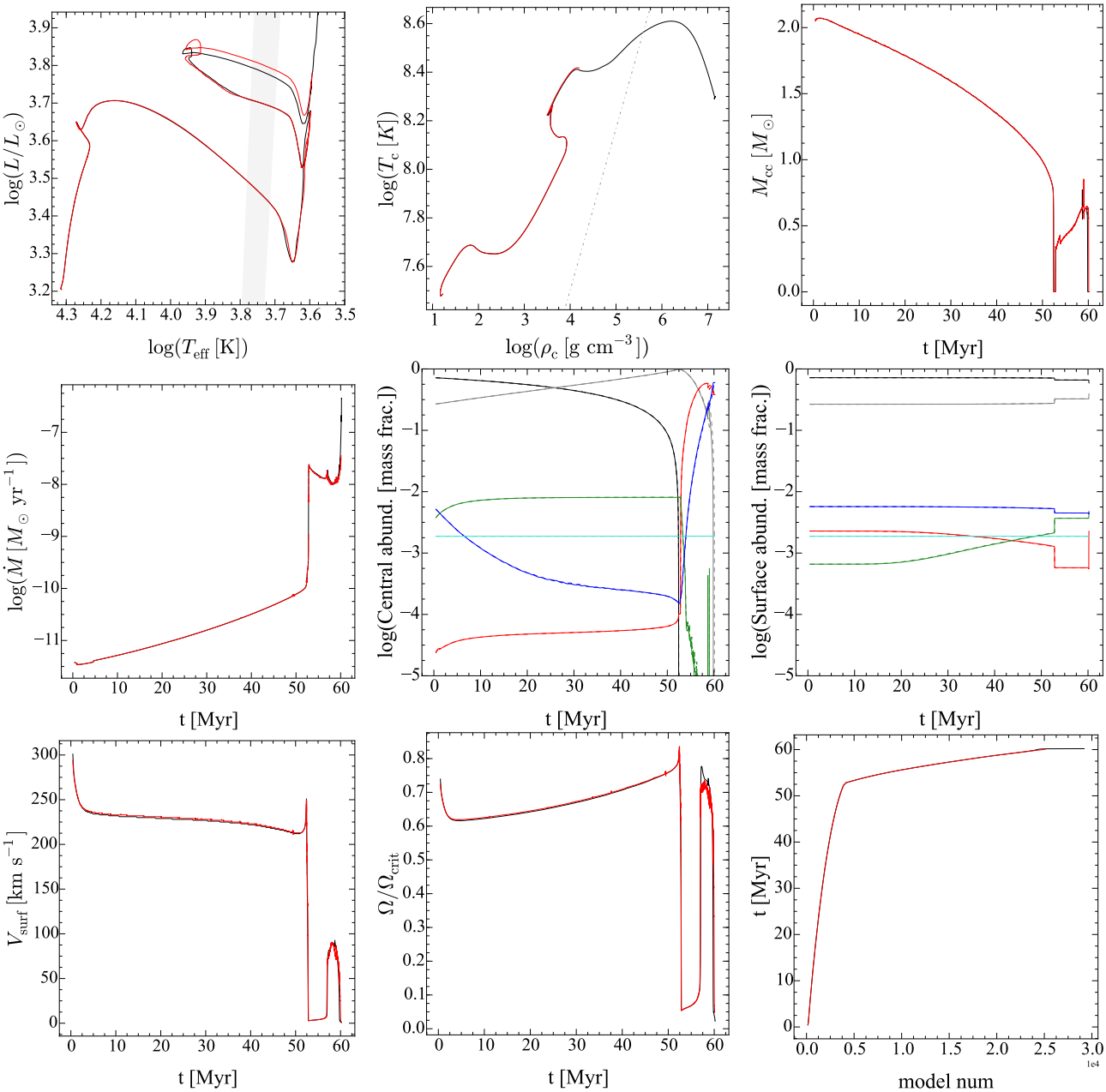


Figure 3: Comparison between the `Origin2013` version of the code (in black, except where explicitly stated) and the new version with reinjection of the residual flux (in red, except where explicitly stated), for a 7 $M_\odot$ star at solar metallicity with an initial rotation rate of $\omega = 0.7$. From top to bottom and left to right: HRD, central density vs. central temperature, convective core mass as a function of time, mass-loss rate as a function of time, central and surface abundances (mass fraction; ^{1}H in black, ^{4}He in gray, ^{12}C in red, ^{14}N in green, ^{16}O in blue, and ^{22}Ne in cyan. `Origin2013` model in solid line, new version in dashed line), surface equatorial velocity as a function of time, $\Omega/\Omega_{\rm crit}$ ratio as a function of time, and time vs timestep.

angular velocity. It is computed to show how is improved the conservation of the angular momentum in the new implementation, without imposing it with the corrections. It is in blue in the plots.

- Eventually, a full implementation discussed in Section 3, including **the re-injection of the residual as an external flux and the correction of the surface angular velocity**. It is shown in cyan in the following plots.

For each of these cases, we computed a $15\,M_\odot$ models at solar metallicity, with an initial rotation velocity of $v_{\rm ini}/v_{\rm crit} = 0.4$. They are computed up to the end of helium burning. We have compared two different kind of outputs: some of them are usual outputs from the stellar evolution code, and some are more technical data, illustrating the convergence of the advection/diffusion schemes, and the amplitude of the surface angular velocity corrections (when applied).

Figures 4 and 5 shows the first set of outputs. In the HRD, the differences are only minor. During the MS, only the red model is slightly different. This is due to a different evolution of the surface velocity, because this model has the less controls of the conservation of the angular momentum (see Fig. 6). The crossing of the HRD is extremely model-dependent, but all models end on a similar red supergiant branch. The central conditions of all the models are extremely similar, and are little affected by what happens at the surface. The only minor difference is between the red model and the other ones: all the models except the red one have a spike near the end of central He-burning (see also the next panel), and this feature is clearly visible on the track at a $\log(T_{\rm c}) \sim 8.3$ and $\log(\rho_{\rm c}) \sim 3.1$. The next panel shows the time evolution of the size of the convective core. All is very similar except again the red model (for the same reasons as discussed above). This model keeps a higher velocity during the MS, reducing the shear, and thus the mixing inside the model. The MS convective core is thus a bit smaller, and the MS has a shorter duration (the size of the core is also responsible for the slight difference in the HRD). The spikes mentioned previously are clearly visible in this plot near the end of central He-burning. The last plot of this first set is the evolution of the effective temperature as a function of the central ^{4}He abundance. This plot exhibits the most sensible behaviour of the models. Indeed, the HRD crossing for all the models are different here. The models in black (`2013`) and in green (similar to the `2013` version in the new implementation) are very similar, because of a very similar treatment of the physics. Residual re-injection as an external flux makes the models cross more quickly the HRD. The red model has an opposite behaviour, with a slower crossing. However, as will be discussed later, this implementation is the less satisfactory.

The first two panels of the next set (Fig. 5) show the time evolution of the ratio $\Omega/\Omega_{\rm crit}$ and equatorial surface velocity. Except for the red model, which do not conserve well the total angular momentum and keeps a higher velocity, all the results are very similar. The central and surface abundances are also in good agreement between all the versions tested here. Finally, the mass-loss rates are behaving similarly, except small differences during the HRD crossing due to the different behaviour in the HRD and in the $T_{\rm eff}$ vs central ^{4}He plot.

One important test for all the implementation is the conservation of angular momentum as a function of time. Let us define the following quantity:

$$\mathcal{L}_{\rm tot}(t) = \mathcal{L}_{\rm star}(t) + \int_0^t \dot{\mathcal{L}}_{\rm wind}(\tau)\mathrm{d}\tau, \tag{133}$$

where $\mathcal{L}_{\rm star}(t)$ is the angular momentum content of the star at the time t, and $\dot{\mathcal{L}}_{\rm wind}(\tau)$ the angular momentum loss due to stellar winds at the time τ. This quantity should remain constant as a function of time, as it includes the angular momentum of the star at a given time, and accounts for all the angular momentum that has been lost by the star in the past.

$\mathcal{L}_{\rm tot}$ is shown on Fig. 6 for the 5 models discussed above. The largest panel shows a large scale view of the obtained behaviour. As expected, the red model (that is computed without re-injection of the residual nor correction of the surface angular velocity) has only a poor conservation of the angular momentum, even with a better treatment of the envelope in this new version. It corresponds to the situation before the introduction of the corrections `CorrOmega` in Georgy (2010). At the end

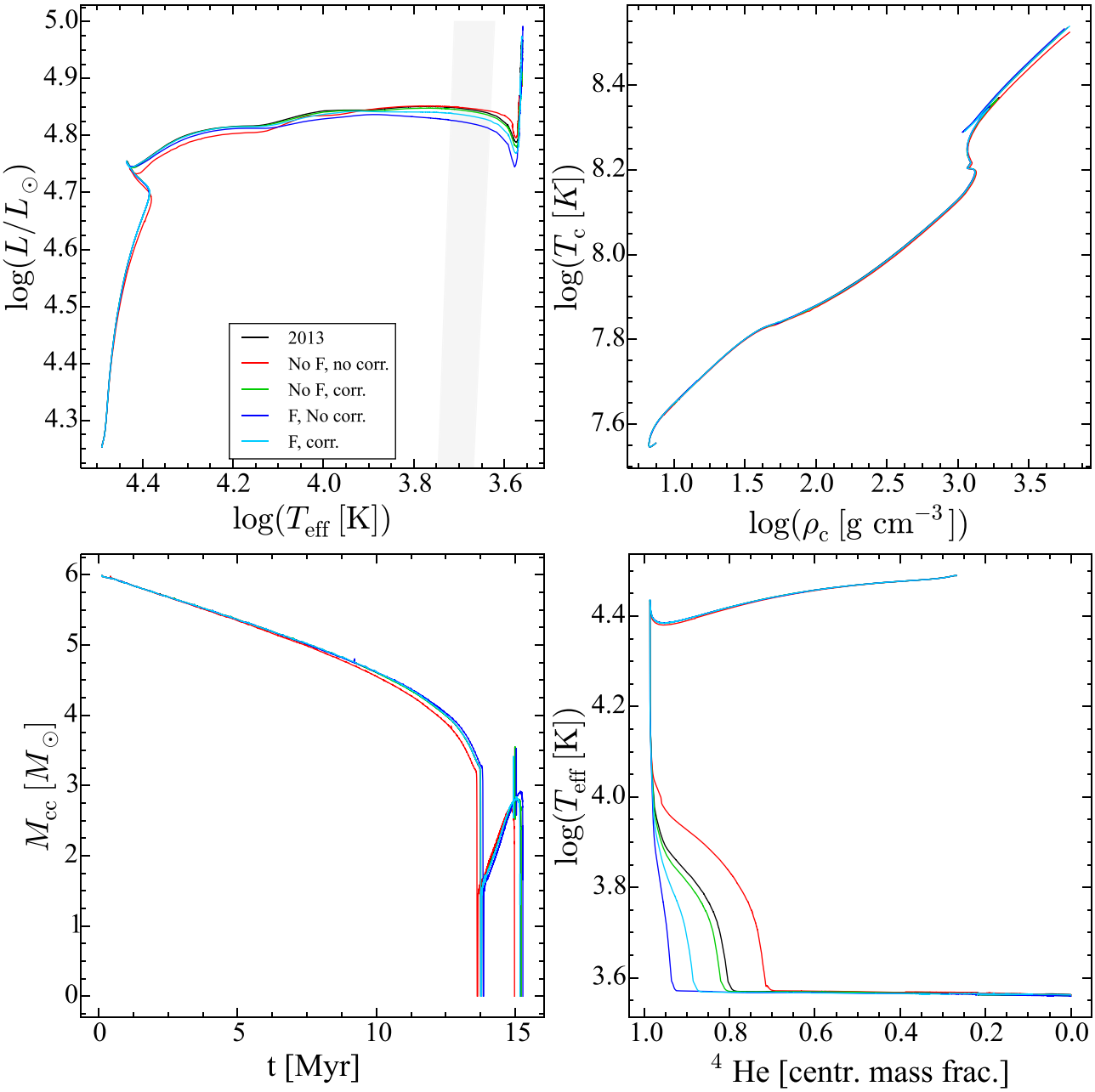


Figure 4: Comparison between the five different implementations discussed in the text. The `2013` version of the code is in black. The other tracks corresponds to the new implementation: the model without re-injection of the residual of the flux and no correction of the surface angular velocity is in red, no re-injection and correction in green, residual re-injection as a flux, but no correction is in blue, and the full new implementation with re-injection of the residual and the correction of the surface angular velocity in cyan. *Top-left panel:* HR diagram, *Top-right panel:* central temperature and density, *Bottom-left panel:* size of the convective core, *Bottom-right panel:* Effective temperature as a function of the central ^{4}He abundance.

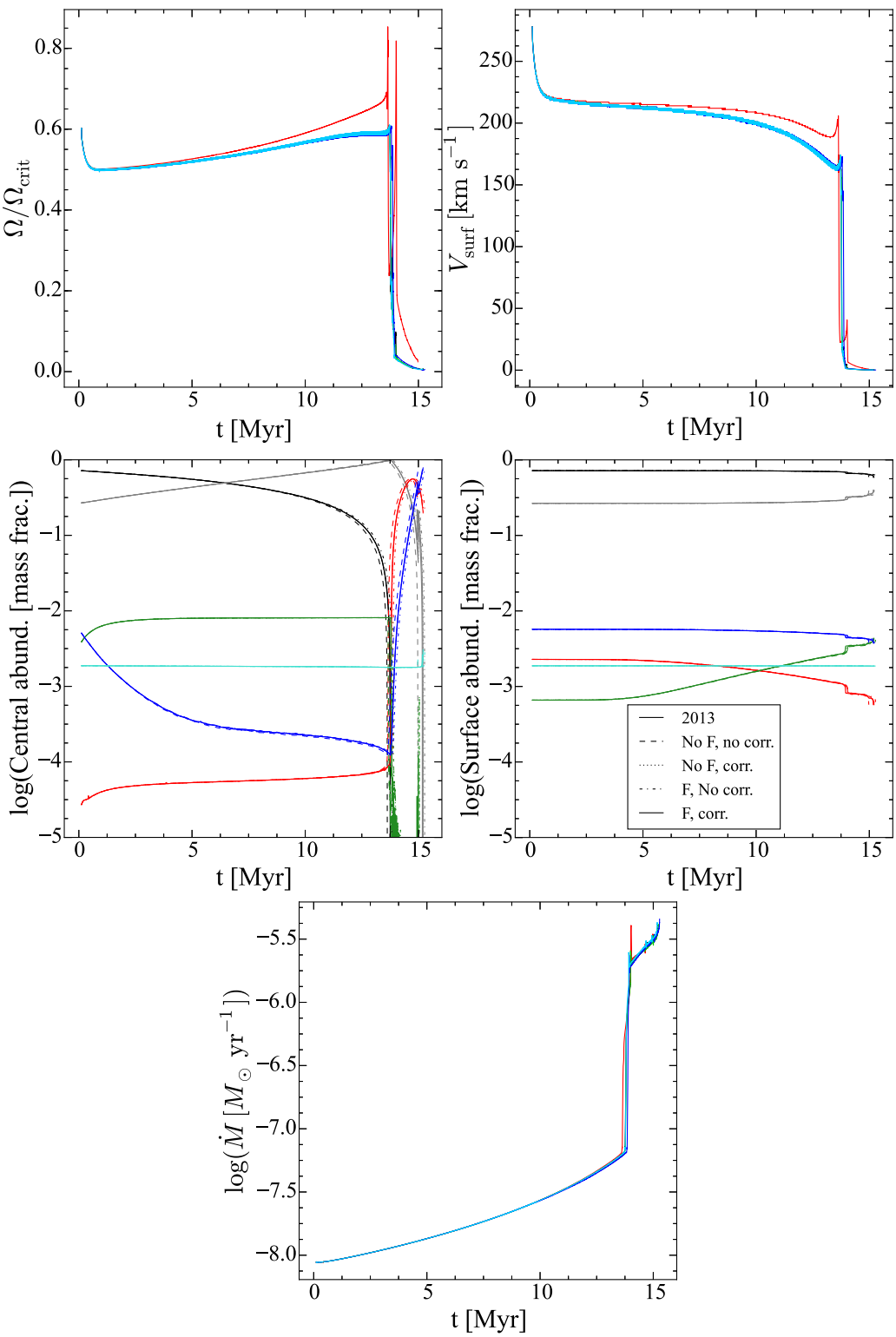


Figure 5: Comparison between the five different implementations discussed in the text. For top and bottom panels, the colour coding is the same as in Fig. 4. In the middle panels, the colours are: ^{1}H (black), ^{4}He (gray), ^{12}C (red), ^{14}N (green), ^{16}O (blue), and ^{22}Ne (cyan). Each tested version of the implementation is represented by a different line style. *Top-left panel:* time evolution of the ratio $\Omega/\Omega_{\rm crit}$, *Top-right panel:* time evolution of the surface equatorial velocity, *Middle-left panel:* time evolution of the central abundances, *Middle-right panel:* time evolution of the surface abundances, *Bottom panel:* time evolution of the mass-loss rate.

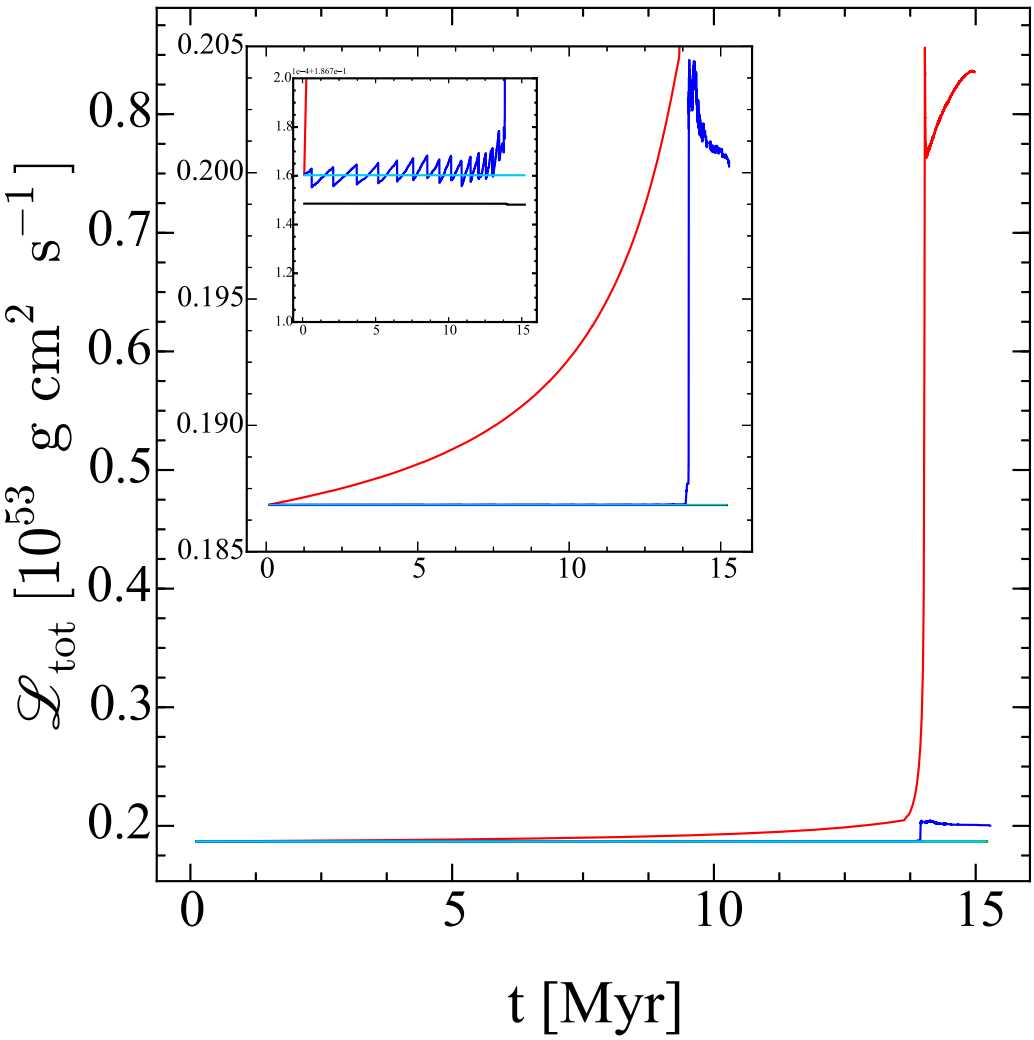


Figure 6: Conservation of the total angular momentum of the system "star + winds" as a function of time. The y-axis is define as $\mathcal{L}_{\rm tot}(t) = \mathcal{L}_{\rm star}(t) + \int_0^t \dot{\mathcal{L}}_{\rm wind}(\tau)\mathrm{d}\tau$. By construction of this quantity, it should remain constant during the course of the evolution.

of central He-burning, the total angular momentum of the system is a factor of more than 4 higher than the initial one. This is why the surface velocity of the red model is systematically larger than the other ones. The divergence occurs mostly during the crossing of the HRD. The first inner panel shows a zoom in the y-axis. It shows that the model computed with the re-injection of the residual, but *without* surface angular velocity corrections (in blue) has a quite good conservation of the total angular momentum of the system. Again, the critical phase is the quick crossing of the HRD, where most of the divergence occurs. However, contrarily to the red model, the variation is much smaller, about 10% compared to the initial value. Finally, the innermost panel is a deep zoom, showing that models accounting for the correction of the surface angular velocity have indeed a constant $\mathcal{L}_{\rm tot}$ during the evolution (black, green, and cyan curves). The blue model discussed previously has also an excellent behaviour during the MS, with very small variations only.

Figure 7 illustrates two technical points. The left panel shows the conservation of the angular momentum during the computation of the advection or diffusion process. $\mathcal{L}_{\rm beg}$ is the total angular momentum of the star at the beginning of the computation of the advection (in `henadv`) or diffusion (in `diffom`). $\mathcal{L}_{\rm end}$ is the angular momentum at the end of the computation, plus the in- or out-flux used as boundary condition. In a perfect world, both quantity should be equal. The figure shows that:

- except at the very beginning of the computation, when the solid-body rotation is released (see the inserted panel), the conservation of the angular momentum during the advection/diffusion

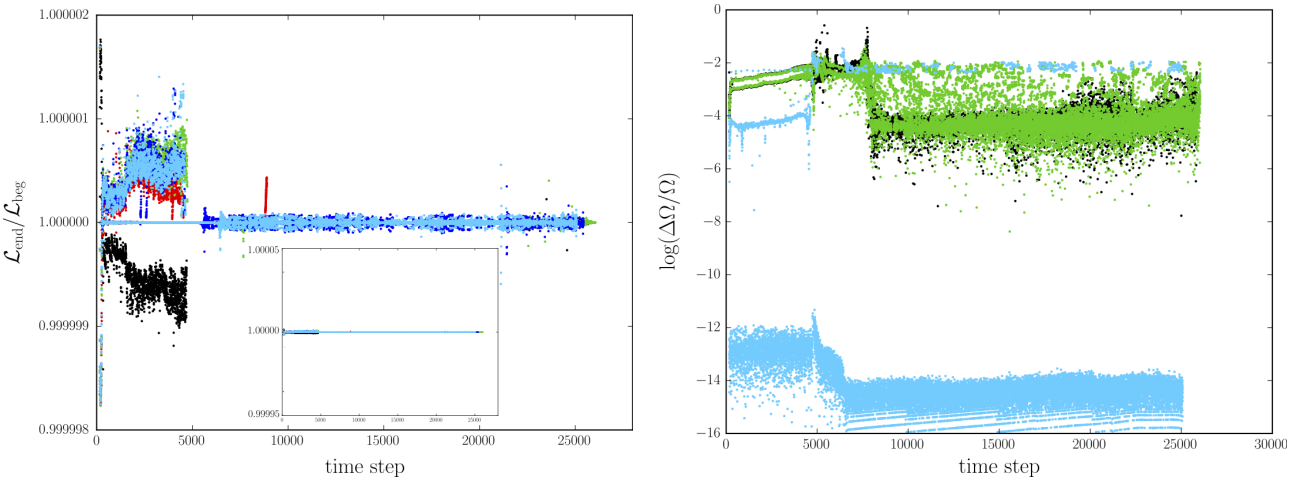


Figure 7: *Left panel:* Conservation of the angular momentum during the computation of the advection/diffusion process. A value of 1 indicates that the final value of the angular momentum is exactly the required one. *Right panel:* Relative value of the correction of the angular momentum applied at the surface of the models where it was accounted for during the computation.

computation is very good (better than $2 \cdot 10^{-6}$. Even considering the very first time step after the ZAMS, it is always better than 10^{-5}.

- During the MS, where advection and diffusion are applied alternatively, we see that the corrections are slightly bigger. This is due to 1) the advection scheme, which is less accurate than the diffusion one, and 2) the correction being applied only each two time steps, it contains half of the time a double contribution.

The conclusion of this test is that the new implementation including the envelope and the new boundary conditions (blue, red, green, and cyan points) is as efficient as the previous implementation of the `2013` version (note that the conservation test shown here for the `2013` version concerns only the interior of the star; the envelope is not considered here. It would be a source of inaccuracy if we had included it in the computation of the angular momentum before and after the transport process). The benefits being that: the envelope is now consistently included in the computation, and the in/out-flux of angular momentum are now included in a more physical way.

The right panel shows the relative correction applied to the angular velocity of the `NPcoucheEff` layers below the surface in case `rapcrilim` is not 0 (so this concerns only the black, green, and cyan models). The black model (`2013` version) and the green one (equivalent implementation in the new version, including envelope and new boundary conditions) are still completely equivalent. The relative corrections during the MS are between 10^{-3} and 10^{-4}. During the crossing of the HRD, they are at about 10^{-2}, and then, for the rest of the simulation, they are most of time at the level of 10^{-4} to 10^{-3}, except a few cases that are around 10^{-2}. When we consider the case of re-injection of the residual as an external flux (cyan points), the corrections become much smaller on average: 10^{-4} on average during the MS for the advective time steps and 10^{-13} to 10^{-12} for the diffusive ones. Only a few exceptions occur, with a correction reaching 10^{-2}. After the MS, the bulk of the corrections are negligible at a level of 10^{-14} or less. A small fraction of the time steps is at the percent level. Generally, in all the implementations, the points that are significantly higher than the average are due to a sudden change in the structure of the envelope. When this occur, the change in the envelope's momentum of inertia is *not* accounted for in the advection/diffusion computation (as it provoques a crash of the code). Instead, it is pass on the corrections, that take care of the angular momentum conservation. The price to pay is that we have much bigger corrections from time to time.

A small comment: Models computed with the re-injection of the residual as an external flux crash from time to time during the main sequence. Usually, relaunching the sequence with a reduced time step is enough to solve the problem. In case we chose the kind of implementation for the future, this change can be easily automated in the python launching script.

References

Chaboyer, B. & Zahn, J. 1992, A&A, 253, 173

Eggenberger, P., Meynet, G., Maeder, A., et al. 2008, Ap&SS, 316, 43

Ekström, S., Georgy, C., Eggenberger, P., et al. 2012, A&A, 537, A146

Ekström, S., Meynet, G., Maeder, A., & Barblan, F. 2008, A&A, 478, 467

Georgy, C. 2010, PhD thesis, Geneva Observatory, Geneva University

Georgy, C., Ekström, S., Granada, A., et al. 2013, A&A, 553, A24

Georgy, C., Meynet, G., & Maeder, A. 2011, A&A, 527, A52

Henyey, L. G., Forbes, J. E., & Gould, N. L. 1964, ApJ, 139, 306

Kawaler, S. D. 1988, ApJ, 333, 236

Kippenhahn, R. & Weigert, A. 1990, Stellar Structure and Evolution (Stellar Structure and Evolution, XVI, 468 pp. 192 figs.. Springer-Verlag Berlin Heidelberg New York. Also Astronomy and Astrophysics Library)

Kudritzki, R.-P. & Puls, J. 2000, ARA&A, 38, 613

Maeder, A. 1997, A&A, 321, 134

Maeder, A. 2002, A&A, 392, 575

Maeder, A. 2003, A&A, 399, 263

Maeder, A. 2009, Physics, Formation and Evolution of Rotating Stars (Springer)

Maeder, A. & Meynet, G. 2000, A&A, 361, 159

Maeder, A. & Meynet, G. 2003, A&A, 411, 543

Maeder, A. & Meynet, G. 2004, A&A, 422, 225

Maeder, A. & Meynet, G. 2005, A&A, 440, 1041

Maeder, A. & Zahn, J.-P. 1998, A&A, 334, 1000

Mathis, S., Palacios, A., & Zahn, J.-P. 2004, A&A, 425, 243

Meynet, G., Eggenberger, P., & Maeder, A. 2011, A&A, 525, L11+

Meynet, G., Maeder, A., & Mowlavi, N. 2004, A&A, 416, 1023

Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. 2007, Numerical recipe. The art of scientific computing (Cambridge: University Press, 2007, 3rd ed.)

Spruit, H. C. 2002, A&A, 381, 923

Talon, S. 1997, PhD thesis, , Observatoire de Paris, (1997), 187 pages

Talon, S. & Zahn, J.-P. 1997, A&A, 317, 749

Ud-Doula, A., Owocki, S. P., & Townsend, R. H. D. 2009, MNRAS, 392, 1022

Zahn, J.-P. 1992, A&A, 265, 115

A Index of some of the variables used for computing rotation

A.1 alpro6

This is the multiplicative factor to be applied to the mass-loss rate in order to account for the rotation:

$$\texttt{alpro6} = \frac{\dot{M}(\Omega)}{\dot{M}(0)}. \tag{134}$$

A.2 az1

`az1` contains the inner boundary condition for computing the advection.

A.3 b1

`b1` contains the external boundary condition for computing the advection.

A.4 bdotis

`bdotis` is the quantity of angular momentum lost during the time step due to radiative stellar winds, when anisotropy is neglected.

A.5 BTotal_EndAdvect

`BTotal_EndAdvect` contains the total angular momentum of the model (including envelope) at the end of advection/diffusion computation, *without* accounting for the re-injection of `Flux_remaining`. It is thus equal to `BTotalStartModel` - `xldoex`.

A.6 BTotalStartModel

`BTotalStartModel` contains the total angular momentum of the converged model of the previous time step, including the integrated envelope with `vsuminenv`. Contrarily to `xLtotbeg`, it is redefined at the beginning of each time step.

A.7 CorrOmega

`CorrOmega` is the correction to be applied on a given number of shells below the surface, in order to ensure the conservation of angular momentum. This correction is applied only each two time steps (when diffusion is applied)[11], so depending on the parity of the time step, it contains the correction for either the current time step, or both the current and previous time step. During the main sequence, the number of layers on which the correction is applied is equal to 200. During the advanced stages, this number is set to 50.

A.8 D_chim

Variable containing the total diffusion coefficient for the chemical species.

[11]This is true for the main sequence. Later on, as advection is no more computed, the correction is applied each time step.

A.9 D_Omega

Variable containing the total diffusion coefficient for angular momentum.

A.10 deladv

deladv contains the difference between the Ω-profile solution of the advection equation omegi at the end of henadv, and the initial profile at the beginning of the structure iteration vvomeg:

$$\texttt{deladv} = \texttt{omegi} - \texttt{vvomeg}. \tag{135}$$

A.11 dL_Kawaler

It contains the angular momentum loss during the current time step due to magnetic braking, according to a Kawaler law (see dLmagLM).

A.12 dlelex

dlelex is the angular momentum loss due to stellar winds, magnetic breaking, mechanical mass loss, *etc.* during the current time step, or two last time steps (depending on when the correction was applied last time).

A.13 dlelexprev

dlelexprev is the angular momentum loss due to stellar winds, magnetic breaking, mechanical mass loss, *etc.* during the previous time step, or zero (depending on when the correction was applied last time).

A.14 dlelexsave

dlelexsave is the lost of angular momentum due to stellar winds, magnetic breaking, mechanical mass loss, *etc.* during the current time step *only*.

A.15 dLmag

dLmag is the angular momentum loss due to magnetic braking *and* radiative stellar winds, as computed in Meynet et al. (2011).

A.16 dLmeca

dLmeca is the angular momentum loss due to mechanical mass loss. It is assumed to be under the form of an equatorial disc.

A.17 dm_lost

dm_lost is the total mass lost during the time step through the radiative stellar winds (in solar masses).

A.18 dmneed

dmneed is an estimate of the equatorial mass loss, needed to keep the star at the critical velocity.

A.19 f

This variable (sometimes called fffff) is the inverse of the oblateness of the star.

A.20 `fitm`

`fitm` is the mass fraction at which the boundary between the interior (where the full set of structure equation is solved) and the envelope (where only three equations are solved, with a different integration variable) is set. In the 2013 version of the code, it is set to 0.9999 when the star is hot enough, and drops to 0.98 when a large convective envelope is present. In the newest version proposed here, it is no more possible to keep such a high value during the main sequence, so it is set to 0.9995 during this stage.

A.21 `Flux_remaining`

`Flux_remaining` contains the difference between the flux that have to be lost during the time step `dlelexsave`, and the angular momentum effectively lost during the time step. This difference is then reinjected as an in/out flux in the boundary conditions of the advection/diffusion computation, to improve the conservation of angular momentum.

A.22 `gms`

`gms` is the total mass of the model, in solar masses.

A.23 `iadvec`

`iadvec` controls the computation of the advection in the code:

0. the advection is not computed,
1. the advection is computed and applied each two time steps (odd numbers).

A.24 `icoeff`

`icoeff` controls the choice of the diffusion coefficients used in the Geneva code to deal with diffusion. This parameter is of the form `icoeff = ab`, where `a` controls the choice of the shear mixing diffusion coefficient: `a=1` for Maeder (1997), and `a=2` for Talon & Zahn (1997), and `b` controls the choice of the horizontal diffusion coefficient: `b=1` for Zahn (1992), `b=2` for Maeder (2003), and `b=3` for Mathis et al. (2004).

A.25 `idialo`

`idialo` manages the computation of a smoothed profile of Ω and $\frac{\mathrm{d}\ln(\Omega)}{\mathrm{d}\ln(r)}$. This is done when it is set to 1 in the parameter card.

A.26 `idialu`

`idialu` manages the computation of a smoothed mean molecular weight gradient. This is done when it is set to 1 in the parameter card.

A.27 `idiff`

`idiff` controls the computation of the diffusion. In case its value is 1, then diffusion is computed for both angular momentum and chemical species transport. Otherwise, it is not applied.

A.28 `imagn`

`imagn` has to be set to 1 when one wants to use the internal magnetic coupling for both chemical species and angular momentum.

A.29 inoadv

inoadv is a parameter managing the computation of the advection. When it is set to 1, advection is not computed during the current iteration.

A.30 K_Kawaler

This variable corresponds to the K parameter appearing in equation (107). In case its value is different from 0, magnetic braking according to Kawaler (1988) is computed. It is set by default to 0.

A.31 newomega

newomega is the estimated surface angular velocity once the mass-loss removal of angular momentum has been accounted for.

A.32 NPcoucheEff

NPcoucheEff is the number of layers immediately below the surface on which the correction of the angular velocity CorrOmega is applied, to ensure the conservation of angular momentum. Its value is 200 during the main sequence, and 50 for the advanced stages.

A.33 Omega_saturation

Omega_saturation contains the value of the free parameter Ω_{sat} appearing in the Kawaler (1988) law for magnetic braking. It is expressed in unit of solar surface angular velocity. Its default value is 14.

A.34 omegd

omegd contains the new Ω-profile, applying the diffusion to the initial profile vvomeg.

A.35 omegi

omegi is the angular velocity. Depending on where it is located inside the code, it represents either the value at the current time step, or the estimated value for the current or next time step.

A.36 omegp

omegp contains the new Ω-profile, assuming local conservation of the angular momentum.

A.37 omlim

omlim contains the limit angular velocity, corresponding to a rotation at a fraction rapcrilim of the critical velocity.

A.38 q

q is a modified mass coordinate inside the star, defined as:

$$q = \ln\left(1 - \frac{M_r}{M_{\text{tot}}}\right). \tag{136}$$

A.39 r

r is the natural logarithm of the radius coordinate. Depending on where it is located inside the code, it represents either the converged value of the previous time step, or the value computed during the current time step.

A.40 rb

rb is the natural logarithm of the radius coordinate of the previous *iteration* in henyey.

A.41 rapcrilim

rapcrilim is a parameter (that can be defined in the parameter card) defining the maximal fraction of the critical velocity that a star is allowed to reach before the mechanical mass loss is switched on. By default, it is set to 0.99. In case it is set to 0, the mechanical mass loss is *not* accounted for, and moreover, *this deactivates the computation and application of the corrections of the angular velocity CorrOmega*. Setting rapcrilim to 0 is thus equivalent to compute a model as it was done in previous versions of the Geneva code.

A.42 rprov

rprov contains an estimate of the radius at the next time step, by linearly extrapolating from the radii at the previous two time steps.

A.43 suminenv

suminenv contains the converged value of the momentum of inertia of the envelope of the current time step.

A.44 vsuminenv

vsuminenv is the momentum of inertia of the envelope of the converged model of the previous time step. It si computed with the radial structure of the envelope at the time it is integrated. It is not used in the 2013 version of the code (a simplified expression is used instead, computed on the fly), but it is in the newest version proposed in this document.

A.45 thext2

thext2 is the boundary condition on Θ at the outer edge of the star.

A.46 vomegi

vomegi is the angular velocity. Depending on where it is located inside the code, it represents either the converged value at the current time step, or the converged value of the previous time step.

A.47 vr

vr is the natural logarithm of the radius coordinate. Depending on where it is located inside the code, it represents either the converged value of the antepenultimate time step, or the value at the previous time step.

A.48 vvomeg

vvomeg contains the converged Ω-profile at the end of the previous time step. This profile is then homogenised in the convective zones of the *current* iteration in henyey.

A.49 vxmoci

vxmoci contains the angular momentum of each layers. Depending on where we are in the code, it is computed using different radial structures and Ω-profiles.

A.50 **xbext**

xbext contains the momentum of inertia (computed in momcon) of the surface convective zone if any.

A.51 **xcext**

xcext contains the angular momentum (computed in momcon) of the surface convective zone if any.

A.52 **xbint**

xbint contains the momentum of inertia (computed in momcon) of the first convective zone below the surface (not included) if any.

A.53 **xcint**

xcint contains the angular momentum (computed in momcon) of the first convective zone below the surface (not included) if any.

A.54 **xldoex**

xldoex contains the angular momentum loss of the current time step, from which the isotropic contribution has been removed. It is used as external flux of angular momentum, as we assume that the mass brought away by the radiative stellar winds and deduced from the stellar mass decreases the total angular momentum of the star of the isotropic part:

$$\texttt{xldoex} = \texttt{dlelex} - \Delta\mathcal{L}_{\text{iso}}. \tag{137}$$

A.55 **xlexcs**

xlexcs is the excess or lack of angular momentum when anisotropic stellar winds are accounted for in the computation. One have:

$$\texttt{xlexcs} = \frac{\Delta\mathcal{L}_{\text{aniso}}}{\Delta\mathcal{L}_{\text{iso}}} - 1, \tag{138}$$

where $\Delta\mathcal{L}_{\text{aniso}}$ is the angular momentum loss when anisotropic winds are accounted for, and $\Delta\mathcal{L}_{\text{iso}}$ the angular momentum loss when the wind is assumed to be isotropic.

A.56 **xLstarbefHen**

xLstarbefHen contains the total angular momentum of the *interior* of the star (in the 2013 version) or of the whole star in the new proposed version, before the diffusion/advection routines are applied. It is used as a check of the angular momentum conservation during the successive iterations of the computation of the new Ω-profile.

A.57 **xLtotbeg**

xLtotbeg is the total angular momentum of the star (including the envelope) used as a reference for the angular momentum conservation. Due to the fact that the correction (see CorrOmega) is applied only each two time steps, this reference is taken either at the beginning of the current time step, or at the beginning of the previous one.

A.58 **xmdot**

Logarithm (base 10) of the mass-loss rate, in solar mass per year.

A.59 `xmocin`

In the routines computing the advection, `xmocin` contains the total angular momentum of each convective zones, computed with the radial structure stored in `vr` and the angular velocity `vomegi` (both of them being the converged values at the previous time step).

A.60 `xMoCinScale`

`xMoCinScale` contains the angular momentum of each layer of the model at the end of the routine `advect`, computed with the radial structure of the previous iteration `rb` and the angular-velocity profile `omegi`.

A.61 `xmomin`

In the routines computing the advection, `xmomin` contains the total momentum of inertia of each convective zones, computed with the radial structure stored in `vr` (converged value at the previous time step).